Boundary Value Analysis, Equivalence Partitioning, and Mutation Testing: Two Multi-Variable Case Studies

Mai Hải Thành[a]

[a]Independent Research; FIT-UET-VNU, Hanoi, Vietnam

September 2026

Contents

1 Introduction **3**

2 Theoretical Background **3**
2.1 Equivalence Partitioning . . . 3
2.2 Boundary Value Analysis . . . 4
2.3 Nominal Values . . . 4
2.4 Multi-Variable Boundary Analysis . . . 4
2.5 Mutation Testing . . . 4

3 Methodology **5**
3.1 From Specification to Domain . . . 5
3.2 Equivalence Partitioning . . . 5
3.3 Boundary Identification . . . 5
3.4 Boundary Combinations . . . 5
3.5 Test Suite Construction . . . 5
3.6 Mutation Analysis . . . 6
3.7 Executed Validation . . . 6

4 Case Study I: ATM Withdrawal Validation **7**
4.1 Problem Context . . . 7
4.2 Formal Specification . . . 7
4.3 Input and Output Domains . . . 8
4.4 Constraints and Dependencies . . . 8
4.5 Nominal Values . . . 9
4.6 Equivalence Partitioning . . . 9
4.7 Partition Diagrams . . . 10
4.8 Boundary Value Analysis . . . 11
4.9 Boundary Combinations . . . 13
4.10 Test Suite Construction . . . 15
4.11 Mutation Model . . . 19
4.12 Test–Mutant Analysis . . . 20
4.13 Executed Validation . . . 22
4.14 Case Study Discussion . . . 23

5 Case Study II: Quadratic Equation Solver **23**
5.1 Problem Context . . . 23
5.2 Formal Specification . . . 23
5.3 Input and Output Domains . . . 24
5.4 Constraints and Dependencies . . . 24
5.5 Representative and Nominal Configurations . . . 25
5.6 Equivalence Partitioning . . . 25
5.7 Partition Diagrams . . . 26
5.8 Boundary Value Analysis . . . 27
5.9 Boundary Combinations . . . 29
5.10 Test Suite Construction . . . 29
5.11 Mutation Model . . . 33
5.12 Test–Mutant Analysis . . . 33
5.13 Executed Validation . . . 35
5.14 Case Study Discussion . . . 36
5.14.1 Comparison with Case Study I . . . 36
5.14.2 Lessons Common to Both Case Studies . . . 37

6 Limitations **39**

7 Conclusion **40**

1 Introduction

Software testing is not the act of running a program and observing that it does not crash; it is the act of *designing* a finite set of inputs that can stand in, with justified confidence, for the infinite set of inputs the program will actually see. Three complementary techniques make that justification possible: Equivalence Partitioning (EP), which divides an input domain into regions the specification treats identically; Boundary Value Analysis (BVA), which recognises that specifications are almost always violated first at the edges of those regions, not in their interior; and Mutation Testing, which turns the question "is this test suite good enough?" into a falsifiable experiment by seeding small, deliberate faults and checking whether the suite notices.

This report applies all three techniques, end to end, to two case studies that are structurally different but methodologically comparable:

(1) **ATM Withdrawal Validation** — a five-input business-rule validator over a mixed numeric/categorical domain, with independent constraints and one arithmetic interaction (fee) that couples two categorical and one numeric input.

(2) **Quadratic Equation Solver** — a three-input mathematical classifier over $a, b, c \in \mathbb{Z}$, whose real behavioural boundary is not any individual coefficient but a *derived* algebraic quantity, the discriminant $\Delta = b^2 - 4ac$.

The report is first a **test-design document**. Every artifact in Sections 4 and 5 – partitions, boundaries, test cases, mutants – is derived analytically from the stated specification, and where mutation killing is discussed there it is explicitly labelled as an *analytically expected* outcome (reasoned from the mutant's semantics and the test's expected output). Each case study then closes with an executed validation (§4.13, §5.13): the specification was implemented, every designed test case was executed against the implementation, the analytically derived mutation matrix was re-computed by executing mutant models, and an automated mutation tool was run to obtain a measured mutation score. Analytical claims and measured results are kept separate: the analytical subsections retain their labelling, and every measured number appears in an executed-validation subsection.

The guiding question behind every test case in this report is:

> *Why does this test case exist?*

The answer is always traceable to one of: an equivalence partition, an individual-variable boundary, a derived (multi-variable) boundary, or an explicit interaction between constraints. Section 2 lays the theoretical groundwork; Section 3 describes the methodology (the pipeline from specification to mutation analysis) in the abstract; Sections 4 and 5 apply it concretely to the two case studies, each ending with its executed validation and a discussion; the discussion of Section 5 also compares the two case studies (§5.14.1) and draws the lessons common to both, including why single-variable testing under-covers interacting systems (§5.14.2); Section 6 states limitations; Section 7 concludes.

2 Theoretical Background

2.1 Equivalence Partitioning

Equivalence Partitioning [1, 5] assumes that a well-specified program treats all members of some subset of its input domain identically with respect to the observable output. Formally, for an input domain D and a specification that induces an output function $f : D \to O$, an equivalence class is a subset $P \subseteq D$ such that for all $x, y \in P$, the specification predicts $f(x)$ and $f(y)$ fall into the same output category. A partitioning $\{P_1, \ldots, P_k\}$ of D is *sound* if every P_i is

behaviourally homogeneous in this sense, and is *complete* (exhaustive) if $\bigcup_i P_i = D$. Classes are further labelled *valid* (satisfy all preconditions) or *invalid* (violate at least one). A single representative value drawn from each class is assumed — under the homogeneity assumption — to exercise the same code path as every other member of that class, which is what allows a finite test suite to stand for an infinite domain.

2.2 Boundary Value Analysis

Boundary Value Analysis (BVA) [1, 2] is motivated by the observation that the homogeneity assumption of EP is only as good as the partitioning itself, and real specifications are overwhelmingly expressed as inequalities ($\leq$, $<$, $\geq$, $>$) and equalities. Faults cluster at the exact points where such a relation flips truth value — an off-by-one error in '$\leq$' versus '$<$' is invisible to a test drawn from the interior of a class but is caught immediately by a test drawn from the point of transition itself. BVA therefore refines EP: for every boundary between two partitions defined by a relation $x \bowtie c$ (with $\bowtie \in \{<, \leq, =, \geq, >\}$ and c the boundary constant), BVA prescribes testing c itself and its immediate neighbours on both sides ($c-1$ and $c+1$ for a discrete domain, or the nearest representable neighbours for a discrete but non-unit-step domain).

2.3 Nominal Values

A nominal ("Norm") value is a value drawn from deep inside a valid partition, chosen so that it is unaffected by any nearby boundary and is therefore unambiguously representative of "ordinary, unremarkable" behaviour for that partition. Nominal values serve two purposes: they act as the baseline against which every boundary or combination test varies exactly one dimension, and they populate the "other" variables in a boundary test so that only the variable under study is actually at its edge. When a specification defines several structurally distinct valid behaviours (as the quadratic solver does: quadratic vs. linear vs. degenerate), a single nominal tuple cannot represent all of them, and multiple baseline configurations must be defined (Section 5.5).

2.4 Multi-Variable Boundary Analysis

Many specifications gate behaviour not on an individual variable but on a *relation among several variables* – e.g., *balance* $\geq$ *amount* + *fee*, or $\Delta = b^2 - 4ac$. Call such a relation a *derived boundary*. A derived boundary cannot be tested by varying one variable at a time while holding the others at nominal values, because the relation's truth value depends on the *joint* state of the variables involved: two variables can each individually be "nominal" for their own one-dimensional partition and yet, together, land exactly on a derived boundary (or violate a different constraint entirely). Multi-variable BVA therefore constructs test tuples where two or more variables are simultaneously placed at values that make a specific joint relation true, false, or exactly zero (the boundary itself), and additionally exercises combinations where changing one variable is shown to change the effective boundary of another (Sections 4.9 and 5.9).

2.5 Mutation Testing

Mutation Testing [3, 4] evaluates a test suite's *fault-detection strength* rather than its code coverage. A *mutant* is the specification (or its implied implementation) with one small, syntactically local fault seeded into it – a relational operator swapped ($\geq \to >$), a boundary constant perturbed (20,000,000 $\to$ 19,999,999), a logical connective flipped (AND $\to$ OR), or an arithmetic term altered ($b^2 - 4ac \to b^2 - 2ac$). A single test *kills* a mutant if the mutant's expected output for that test's input differs from the original specification's expected output for the same input, and that test *survives* the mutant otherwise (same output under both). A mutant *survives the suite* if every test in the suite survives it individually, i.e. no test in the suite kills it. Mutation score is defined as

$$\text{Mutation Score} = \frac{\text{Killed Non-Equivalent Mutants}}{\text{Total Non-Equivalent Mutants}} \times 100\%.$$

An *equivalent mutant* is one that, despite being syntactically different, is semantically identical to the original for every input in the domain (no test can ever kill it, and it must be excluded from the score's denominator). Sections 4 and 5 treat all mutant-killing claims as **analytically expected**, derived from reasoning about the mutant's semantics against each test's specified expected output; measured mutation results, obtained by executing the mutants, are reported separately in §4.13 and §5.13.

3 Methodology

Both case studies were designed by walking the same six-stage pipeline (§3.1–3.6), and both designs were then validated by a seventh, executed stage (§3.7).

3.1 From Specification to Domain

Each case study begins with an informal problem statement, which is first made precise as a *formal specification*: a function signature (named inputs with types), a set of explicit constraints (inequalities, equalities, categorical memberships), and an explicit, enumerated output domain. Every input's domain D_i is stated explicitly (e.g. $amount \in \mathbb{Z}$, $accountStatus \in \{\text{Active}, \text{Frozen}\}$), and the joint valid region $R \subseteq D_1 \times \cdots \times D_n$ is defined as the subset satisfying every stated constraint simultaneously.

3.2 Equivalence Partitioning

For every input, and for every derived quantity the specification depends on (fee, discriminant, $dailyUsed + amount$), the domain is split into mathematically defined equivalence classes, each labelled valid or invalid, with an explicit statement of why each class is internally behaviourally homogeneous. Partition completeness (do the classes cover D_i?) and mutual exclusivity (do they overlap?) are checked explicitly.

3.3 Boundary Identification

For every partition boundary identified in Stage 2 – both direct (single-variable, e.g. $amount = 50{,}000$) and derived (multi-variable, e.g. $dailyUsed + amount = 20{,}000{,}000$, or $\Delta = 0$) – the exact boundary value and its neighbours are identified. For discrete domains where the underlying quantity is not densely representable (Section 5.8's discriminant reachability argument), the neighbours used are the nearest *reachable* values, not a blind ± 1.

3.4 Boundary Combinations

Boundaries are not tested one variable at a time only. Stage 4 explicitly constructs tuples in which two or more variables are simultaneously placed at or near their respective boundaries, including cases where one variable's value changes which threshold another variable is being tested against (the internal/external fee changing the effective balance boundary; a coefficient change crossing $\Delta = 0$ while another coefficient stays fixed).

3.5 Test Suite Construction

Candidate tests are generated from partitions (Stage 2), from boundaries (Stage 3), and from boundary combinations (Stage 4); candidates that are subsumed by another candidate (same partition/boundary coverage, no new information) are removed; the retained set is the final

suite. Every retained test carries an explicit traceability record: which partition, boundary, and/or interaction it targets, and the expected output.

3.6 Mutation Analysis

Realistic mutants – one per plausible implementation slip suggested by the specification's own operators and constants – are defined for each case study, and each mutant is checked against the final test suite from Stage 5, producing a Test×Mutant matrix (`K`/`S`/`-`) that is used to argue the suite's adequacy and to flag any surviving mutant as evidence of a missing test.

3.7 Executed Validation

Stages 1–6 produce a design whose mutant-killing claims are *analytically expected*. A seventh, executed stage checks that design against an implementation of the specification. Both specifications were implemented in C# on .NET 10 (SDK 10.0.401, runtime 10.0.12, Windows 11). Tests were written with xUnit.net v3 (4.0.1) on the Microsoft Testing Platform, property-based tests with FsCheck 3.4.0, and mutation testing used Stryker.NET 5.0.0[1]. Every number reported in §4.13 and §5.13 comes from the reports of one end-to-end run on 2 October 2026 (clean Release build, compiler warnings treated as errors); none is estimated. The executed stage consists of four checks per case study:

(i) **Suite execution.** Each designed test case is one data row that carries its identifier (TC-ATM-*nn*, TC-QE-*nn*) and its partition, boundary and combination columns as test metadata, so a run can be filtered by, e.g., combination C9 alone. Supplementary tests cover values that the boundary analysis names but does not retain, and the implementation's handling of inputs outside the modelled domain.

(ii) **Matrix reproduction.** Each designed mutant is implemented in an executable reference model of the specification that injects exactly one mutant. The model is first checked to agree with the specification and the implementation on every suite row. For each mutant, the set of suite tests whose output differs from the unmutated model is then compared with the set of `K` cells in that mutant's column of the analytical matrix, transcribed verbatim.

(iii) **Tool-generated mutants.** Stryker.NET generates mutants from the implementation's source code with its standard operators. Unlike the designed mutants, this population was not constructed together with the suites.

(iv) **Properties and exhaustive execution.** FsCheck checks invariants and metamorphic relations on 1,000 (ATM) or 2,000 (quadratic) generated cases per property; where the domain is finite (quadratic), the classifier is executed on all of it.

Table 1 lists the resulting test layers.

[1] https://stryker-mutator.io

Table 1: Executed test layers (number of tests per case study)

Layer	Content	ATM	Quadratic
Designed suite	TC-ATM-01–22; TC-QE-01–16 (TC-QE-08 as two sequence checks)	22	17
Supplementary	Values named but not retained in §4.8/§5.8, domain guards, overflow, undefined status	18	26
Matrix reproduction	Executable mutants M1–M14 / N1–N10 against the designed suite	39	28
Properties	Invariants and metamorphic relations (FsCheck)	8	6
Exhaustive	Every $(a, b, c) \in [-100, 100]^3$	–	2
Total		87	79

4 Case Study I: ATM Withdrawal Validation

4.1 Problem Context

An ATM must decide, for a single withdrawal request, whether to approve it or reject it, and if rejecting, for which single, well-defined reason. The decision depends on five inputs that interact: the account's monetary state (current balance, amount already withdrawn today), the requested amount, the account's administrative state (active or frozen), and whether the transaction is being made at the bank's own ("internal") ATM network or a foreign ("external") one, which determines whether a fee is charged.

4.2 Formal Specification

$$\mathit{Withdraw}(\mathit{balance},\ \mathit{dailyUsed},\ \mathit{amount},\ \mathit{accountStatus},\ \mathit{isInternal}) \to D_{out} \tag{1}$$

with input domains

$$\mathit{balance} \in \mathbb{Z}, \quad \mathit{dailyUsed} \in \mathbb{Z}_{\geq 0}, \quad \mathit{amount} \in \mathbb{Z},$$
$$\mathit{accountStatus} \in \{\text{Active}, \text{Frozen}\}, \quad \mathit{isInternal} \in \{\text{True}, \text{False}\}.$$

Assumption (stated explicitly, since the specification leaves it open). $\mathit{balance} \in \mathbb{Z}$ is modelled as an unbounded (signed) integer, since an account may legitimately be overdrawn and no explicit upper bound is given; for concrete test values we use representative magnitudes that stay within ordinary 32-bit signed range. *dailyUsed* is refined to $\mathit{dailyUsed} \in \mathbb{Z}_{\geq 0}$ rather than left as all of $\mathbb{Z}$: it represents money already withdrawn earlier the same calendar day by a running accumulator, and no process that only ever adds non-negative withdrawal amounts to a zero-initialised counter can produce a negative value, so $\mathit{dailyUsed} < 0$ is outside the intended behavioural model and is excluded from the domain by construction rather than treated as a distinct rejection case.

Business rules.

$$\text{(R1) Status:} \quad \mathit{accountStatus} = \text{Active} \tag{2}$$

$$\text{(R2) Amount range \& granularity:} \quad 50{,}000 \leq \mathit{amount} \leq 20{,}000{,}000$$
$$\wedge \ \mathit{amount} \equiv 0 \pmod{50{,}000} \tag{3}$$

$$\text{(R3) Daily limit:} \quad \mathit{dailyUsed} + \mathit{amount} \leq 20{,}000{,}000 \tag{4}$$

$$\text{(R4) Fee:} \quad \mathit{fee} = \begin{cases} 0, & \mathit{isInternal} = \text{True} \\ 50{,}000, & \mathit{isInternal} = \text{False} \end{cases} \tag{5}$$

$$\text{(R5) Balance:} \quad \mathit{balance} \geq \mathit{amount} + \mathit{fee} \tag{6}$$

Output domain.

$$D_{out} = \{\text{Approved},\ \text{Rejected_InvalidAccount},\ \text{Rejected_InvalidAmount},\ \text{Rejected_DailyLimit},\ \text{Rejected_InsufficientBalance}\}$$

Precedence rule (mandatory, since several rules may fail simultaneously). The validator evaluates rules in this fixed order and returns on the first failure:

$$\text{R1} \rightarrow \text{R2} \rightarrow \text{R3} \rightarrow \text{R5} \rightarrow \text{Approved}.$$

Rationale: account status is an administrative precondition and is checked before any monetary reasoning is meaningful (a frozen account's amount is irrelevant); amount well-formedness (R2) is checked next because daily-limit and balance arithmetic are only meaningful for a well-formed amount; the daily limit (R3) is a policy ceiling independent of the customer's own funds and is checked before the customer-specific balance check (R5), reflecting that daily-limit rejection is a bank-side policy stop that should not depend on, or leak information about, the account balance. This fixed order removes all ambiguity about which single rejection reason is returned when multiple rules fail at once, and it is exercised explicitly by TC-ATM-16 in Section 4.9/4.10, the only test in the suite where R3 and R5 fail *simultaneously* (see the discussion of combination C4 and mutant M11).

4.3 Input and Output Domains

Table 2: ATM input domains

Input	Domain	Role
balance	$\mathbb{Z}$	Current account balance (currency units)
dailyUsed	$\mathbb{Z}_{\geq 0}$	Amount already withdrawn earlier today
amount	$\mathbb{Z}$	Requested withdrawal amount
accountStatus	$\{\text{Active}, \text{Frozen}\}$	Administrative account state
isInternal	$\{\text{True}, \text{False}\}$	Own-network vs. foreign-network transaction

D_{out} as defined above is a partition of all possible decisions into five mutually exclusive, collectively exhaustive categories (Section 4.6 shows the mapping from every input-space region to exactly one of these).

4.4 Constraints and Dependencies

The five rules are **not** independent single-variable checks: R4–R5 couple *isInternal* (categorical) with *amount* and *balance* (numeric) through the derived quantity *fee*; R3 couples *dailyUsed* and *amount* through their sum. The joint valid region is

$$R_{\text{ATM}} = \Big\{(b, d, a, s, i) \in \mathbb{Z} \times \mathbb{Z}_{\geq 0} \times \mathbb{Z} \times \{\text{Active}, \text{Frozen}\} \times \{\text{T}, \text{F}\} \;\Big|\; \text{R1–R5 all hold}\Big\}.$$

4.5 Nominal Values

Table 3: ATM nominal (Norm) configuration

Input	Nominal value	Justification
balance	1,000,000	Far above any boundary; comfortably covers amount+fee
dailyUsed	0	Domain minimum of $\mathbb{Z}_{\geq 0}$ (start-of-day value), but far from the R3 boundary at 20,000,000
amount	500,000	Valid multiple of 50,000, far from both R2 endpoints
accountStatus	Active	The single valid class of R1
isInternal	True	Simpler ($fee = 0$) baseline; the External baseline is tested separately (§4.9)

Under this tuple all rules R1–R5 hold, so the nominal case is expected to return Approved (TC-ATM-01).

4.6 Equivalence Partitioning

*amount***.**

$$P_{a,1} = \{x \in \mathbb{Z} \mid x < 50{,}000\} \quad \text{invalid, low (includes } x \leq 0) \quad (7)$$

$$P_{a,2} = \{x \in \mathbb{Z} \mid 50{,}000 \leq x \leq 20{,}000{,}000 \ \wedge \ x \equiv 0 \pmod{50{,}000}\} \quad \text{valid} \quad (8)$$

$$P_{a,3} = \{x \in \mathbb{Z} \mid 50{,}000 \leq x \leq 20{,}000{,}000 \ \wedge \ x \not\equiv 0 \pmod{50{,}000}\} \quad \text{invalid, non-multiple} \quad (9)$$

$$P_{a,4} = \{x \in \mathbb{Z} \mid x > 20{,}000{,}000\} \quad \text{invalid, high} \quad (10)$$

These four classes are mutually exclusive and their union is $\mathbb{Z}$ (exhaustive): every integer falls in exactly one, since $P_{a,2}$ and $P_{a,3}$ partition the closed range by the modulo condition and $P_{a,1}$, $P_{a,4}$ cover everything outside that range.

balance **(relative to a fixed threshold** $T = amount + fee$**).**

$$P_{bal,1} = \{x \in \mathbb{Z} \mid x < T\} \quad \text{invalid, insufficient} \quad (11)$$

$$P_{bal,2} = \{x \in \mathbb{Z} \mid x \geq T\} \quad \text{valid, sufficient} \quad (12)$$

This is a *relative* partition: its boundary T moves with *amount* and *isInternal*, which is exactly the multi- variable interaction analysed in §4.9.

dailyUsed + *amount* **(derived quantity).**

$$P_{dl,1} = \{(d, a) \mid d + a \leq 20{,}000{,}000\} \quad \text{valid} \quad (13)$$

$$P_{dl,2} = \{(d, a) \mid d + a > 20{,}000{,}000\} \quad \text{invalid} \quad (14)$$

*accountStatus***.** $P_{s,1} = \{\text{Active}\}$ (valid), $P_{s,2} = \{\text{Frozen}\}$ (invalid) – an exhaustive, mutually exclusive 2-class categorical partition by construction of the domain.

*isInternal***.** $P_{i,1} = \{\text{True}\}$, $P_{i,2} = \{\text{False}\}$ – both valid individually (isInternal never itself invalidates a transaction), but each selects a different *fee* and hence a different effective T for P_{bal}.

Joint valid region.

$$R_{\text{ATM}} \subseteq D_{bal} \times D_{dailyUsed} \times D_{amount} \times D_{status} \times D_{internal}$$

is exactly $P_{s,1} \times P_{a,2} \times P_{dl,1} \times P_{bal,2}$ (with $P_{dl,1}$ and $P_{bal,2}$ evaluated at the chosen *amount* and *isInternal*) – i.e. approval requires membership in the valid class of *every* partition simultaneously, which is precisely why the test suite must include combinations, not five independent single-variable tests.

4.7 Partition Diagrams

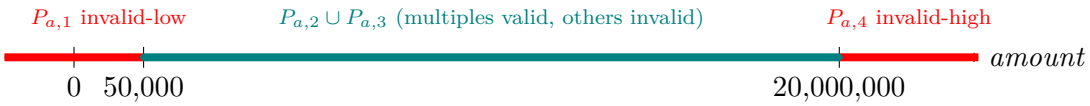


Figure 1: *amount* partitioning on a number line (all other inputs held at their nominal values). Teal tick marks on $[50{,}000, 20{,}000{,}000]$ denote the reachable valid multiples of 50,000; any non-tick integer in that same visual interval belongs instead to invalid class $P_{a,3}$.

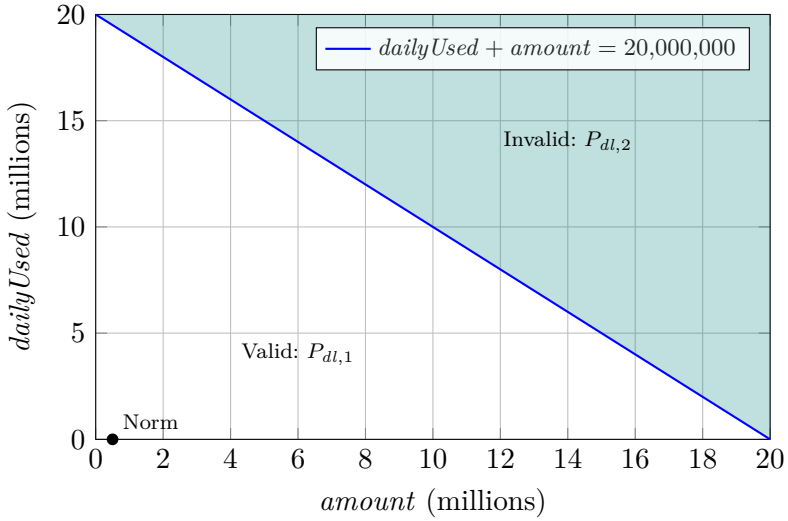


Figure 2: Daily-limit interaction: the derived boundary line $dailyUsed + amount = 20{,}000{,}000$ in the ($amount$, $dailyUsed$) plane, with *balance*, *accountStatus*, *isInternal* fixed at their nominal values. This is a 2D projection of the 5-dimensional input space.

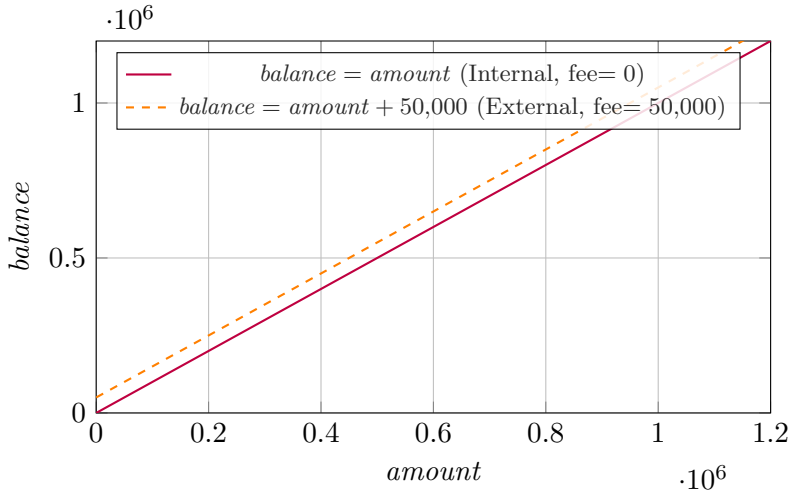


Figure 3: Balance/fee interaction: the effective balance boundary line shifts upward by exactly 50,000 when *isInternal* flips from TRUE to FALSE, with *dailyUsed* = 0, *accountStatus* = Active. This is a multi-variable boundary illustration: the same (*amount*, *balance*) pair can be Approved or Rejected_InsufficientBalance purely depending on *isInternal*.

4.8 Boundary Value Analysis

Amount lower boundary: a discrete range boundary under a granularity constraint ($x = 50{,}000$). R2 is the conjunction of a *numerical range* condition and a *discrete granularity* (divisibility) condition, so its boundary cannot be read as a single continuous inequality edge: 50,000 is not merely "the smallest number $\geq 50{,}000$," it is *the first value that is simultaneously* $\geq 50{,}000$ *and a multiple of* 50,000, i.e. the first *reachable* valid amount. Consequently 49,999 fails range *and* would also fail divisibility if range were somehow relaxed (it is not itself a multiple of 50,000), so it is classified here by range alone under the fixed precedence order (range is checked, and fails, before divisibility would even matter); by contrast 50,001 fails on granularity *alone* (range holds, divisibility fails) even though it is numerically adjacent to a valid point. A value that fails range *purely*, with divisibility never even a candidate distinction, is 0 (TC-ATM-20, §4.10), which is itself a multiple of 50,000 and so isolates the range failure mode cleanly. These are, taken together, genuinely different kinds of failure clustered around the same nominal boundary value, not a symmetric ±1 perturbation of one inequality:

Table 4: *amount* lower boundary — discrete range boundary under a granularity constraint

Range-invalid side	First reachable value	Granularity-invalid side	Nominal	Expected behaviour
49,999 (range fails)	50,000 (valid: range holds *and* divisible)	50,001 (range holds, granularity fails)	500,000	Both invalid sides reject as `InvalidAmount`; only 50,000 is Approved-eligible

amount = 0 is analysed separately (TC-ATM-20, §4.10) as a representative deep-invalid value of $P_{a,1}$ and as the unique test that isolates a pure range failure (above); it is also the value that distinguishes mutant M10 (lower bound removed), since 0 is itself a multiple of 50,000 and would therefore be wrongly accepted if the lower bound were dropped.

Amount upper boundary: the same discrete pattern in reverse (x = 20,000,000). By the same reasoning, 20,000,000 is the *last reachable valid amount*: it is simultaneously $\leq$ 20,000,000 and a multiple of 50,000. Its two neighbours fail for two different reasons: 19,999,999 fails on granularity alone (it is in-range but not a multiple of 50,000), while 20,000,001 fails on *both* range and granularity simultaneously (it exceeds the limit *and* is not a multiple of 50,000), which makes it a poor representative for isolating the "too high" failure mode in particular. The representative over-range value used below is instead 20,050,000 (the next reachable multiple of 50,000 above the limit), which fails on range grounds alone (divisibility holds) and cleanly isolates that failure mode from the granularity failure already exercised at 50,001 (TC-ATM-07) and 60,000 (TC-ATM-18, §4.12); no test in the suite uses 19,999,999 itself, since its role here is illustrative (Table 5), not a retained test case:

Table 5: *amount* upper boundary — discrete range boundary under a granularity constraint

Granularity-invalid side	Last reachable value	Range-invalid side	Expected behaviour
19,999,999 (in range, not a multiple)	20,000,000 (valid: range holds *and* divisible)	20,050,000 (over range, still a multiple)	Both invalid sides reject as `InvalidAmount`; TC-ATM-05 places *balance* = 20,000,000 so that this boundary value is deterministically Approved

Granularity (divisibility) boundary in the interior of the range. Around every reachable multiple $k \cdot 50{,}000$ strictly inside (50,000, 20,000,000), the immediate integer neighbours $k \cdot 50{,}000 \pm 1$ are non-multiples and hence invalid purely on granularity grounds, with range never at issue. TC-ATM-07 (50,001) and TC-ATM-18 (60,000, chosen specifically because it is divisible by 10,000 but not by 50,000, §4.12/M8) both instantiate this interior granularity boundary.

Balance boundary (*balance* = *amount* + *fee*). The three points immediately around the threshold T are:

Table 6: Balance boundary around T = *amount* + *fee*

balance = $T-1$	*balance* = T	*balance* = $T+1$	Expected
Rejected_InsufficientBalance	Approved (boundary satisfies $\geq$)	Approved	R5 is non-strict ($\geq$), so equality is the last valid point, not the first invalid one

Daily-limit boundary (*dailyUsed* + *amount* = 20,000,000). The three points immediately around this derived sum are:

Table 7: Daily-limit boundary

Sum = 19,999,999	Sum = 20,000,000	Sum = 20,000,001
Valid (R3 holds)	Valid (R3 holds, equality)	Invalid (Rejected_DailyLimit)

Status boundary (categorical). Active ↔ Frozen: since D_{status} has exactly two members with no ordering, the "boundary" is simply the exhaustive pairwise test of both values, each otherwise-valid.

Internal/external fee boundary (categorical, but numerically consequential). Flipping *isInternal* while *amount* and *balance* are unchanged can flip the outcome of R5, because it moves the threshold T by exactly 50,000 (Figure 3). This is analysed jointly with the balance boundary in §4.9, since testing *isInternal* alone (holding *amount*, *balance* nominal, both far from T) would never reveal this interaction.

4.9 Boundary Combinations

Single-variable BVA is provably insufficient here: a test with *amount* = 50,000 and a separately-chosen "boundary" *balance* value taken from *that variable's own* boundary table (e.g. *balance* = 999,999/1,000,000/1,000,001, boundaries of an unrelated nominal T') would never place *balance* at the boundary *that actually matters for this amount*, namely $T = 50{,}000 + fee$. The combinations below fix that.

Table 8: Mandatory ATM boundary combinations

Combination	Inputs (others nominal)	Why it matters
C1	*amount* = 50,000, *balance* = 50,000	Amount at its own lower boundary *and* balance at exactly T (Internal) simultaneously: both R2 and R5 are boundary-tight at once.
C2	*amount* = 50,000, *balance* = 50,001	Same amount boundary, balance now one unit past T: isolates that R5 alone flips while R2 stays satisfied.
C3	*balance* = *amount* + *fee* and *dailyUsed* + *amount* = 20,000,000 simultaneously (e.g. *amount* = 500,000, *dailyUsed* = 19,500,000, *balance* = 500,000, Internal)	Two independent derived boundaries (R3 and R5) are both exactly tight in the same test; per the precedence rule, R3 is checked first, so this must still be Approved only if *both* hold, and any single-bit perturbation of either boundary must reject for the correspondingly different reason.

Combination	Inputs (others nominal)	Why it matters
C4	$dailyUsed + amount =$ 20,000,001 *and* $balance < amount + fee$ simultaneously (e.g. $amount =$ 500,000, $dailyUsed =$ 19,500,001, $balance =$ 499,999, Internal)	Both R3 *and* R5 fail at once. This is deliberately *not* the simpler case "R3 fails, R5 would have passed": if only R3 failed, checking R3 before R5 or R5 before R3 would report the same reason either way (R3 fails immediately in the first order; in the second order R5 passes and R3 is then found to fail) – a single-failure case cannot distinguish the two orderings at all. Only a *simultaneous* double failure does: checking R3 first reports Rejected_DailyLimit, checking R5 first reports Rejected_InsufficientBalance. This is therefore the only combination that actually exercises, and can kill, a precedence-swap mutant (§4.12, M11).
C5	$amount =$ 500,000, $balance =$ 500,000, $isInternal =$ True vs. $isInternal =$ False (balance unchanged)	Toggling *isInternal* alone, at a balance sitting exactly on the Internal threshold, must flip the outcome from Approved to Rejected_InsufficientBalance purely because T moved from 500,000 to 550,000 – direct evidence that isInternal and balance are not independently testable.
C6	$isInternal =$ False, $amount =$ 500,000, $balance =$ 520,000	Places *balance* strictly between the correct threshold (550,000) and a plausible wrong-fee threshold (505,000): the correct specification rejects (Insufficient), while a mutant using an incorrect fee constant would approve. Needed because C5's balance (500,000) sits below *both* thresholds and cannot distinguish "fee removed" from "fee has the wrong value."
C7	$amount = 0$, others nominal	The lower range bound and the granularity condition agree that 0 is invalid (it is below 50,000), but 0 is itself a multiple of 50,000; only a mutant that has actually removed or lowered the range floor would accept it, making this the unique test able to kill such a mutant (§4.12, M10).
C8	$isInternal =$ False, $amount =$ 500,000, $balance =$ 580,000	Every other External test in the suite (TC-ATM-14, -19) is Rejected, so no test exercises the External branch's Approved path or distinguishes the correct fee threshold (550,000) from a fee that is too *high* (600,000 if fee were mistakenly 100,000); 580,000 sits strictly between the two, so the correct specification approves while an overcharging-fee mutant would reject (§4.12, M13).

Combination	Inputs (others nominal)	Why it matters
C9	*accountStatus* = Frozen, *amount* = 49,999, others nominal	R1 *and* R2 fail simultaneously. Exactly as C4 does for R3/R5, this is the only configuration able to distinguish the specified R1-before-R2 precedence from a reversed one: when only one of R1, R2 fails (as in every other test), checking either rule first yields the same reported reason, so only a genuinely simultaneous double failure can kill a precedence-swap mutant on R1/R2 (§4.12, M14).

4.10 Test Suite Construction

Candidate tests were generated from: (i) one representative per equivalence class in §4.6 (8 candidates: $P_{a,1..4}$, $P_{s,1..2}$ each combined with otherwise-valid inputs, $P_{i,1..2}$); (ii) one test per boundary row in §4.8 (roughly 12 candidates); (iii) the nine combinations C1–C9 of §4.9, of which C1–C5 were already candidates and C6–C9 were added specifically because no candidate from (i)/(ii) was able to distinguish mutants M7, M10, M13, and M14 respectively (§4.12) – a concrete instance of mutation analysis feeding back into test-suite construction. Redundant candidates (e.g. a plain $P_{a,3}$ representative that coincides with an already-scheduled divisibility-boundary test) were merged. Five tests were added, or repurposed from an existing combination, specifically to close a gap that only the mutation analysis of §4.12 exposed – no candidate from (i)–(iii) could otherwise distinguish the mutant named: TC-ATM-18 (interior granularity at 60,000) for M8, TC-ATM-19 (=C6) for M7, TC-ATM-20 (=C7) for M10, TC-ATM-21 (=C8) for M13, and TC-ATM-22 (=C9) for M14. This is a legitimate part of test-suite construction (mutation analysis feeding back to close a coverage gap), not a change to the specification. The resulting twenty-two tests are the complete, non-redundant ATM suite; every value below was computed directly from the formal specification in §4.2, not asserted informally. The same twenty-two tests are listed below; the coverage of every partition and every boundary by these tests is checked directly after the table.

Table 9: ATM complete test suite, with full traceability

ID	Inputs (*bal* / *used* / *amt* / *status* / *internal*)	Partition	Boundary	Combo	Expected output / Rationale
TC-ATM-01	1,000,000 / 0 / 500,000 / Active / True	$P_{a,2}, P_{bal,2}, P_{dl,1}, P_{s,1}, P_{i,1}$		–	**Approved**. Pure nominal baseline; establishes the default valid path.
TC-ATM-02	1,000,000 / 0 / 49,999 / Active / True	$P_{a,1}$	amount lower boundary, range side (-1)	–	**Rejected_InvalidAmount**. One unit below R2's range floor.
TC-ATM-03	50,000 / 0 / 50,000 / Active / True	$P_{a,2}$	first reachable amount = balance = T	C1	**Approved**. Two boundaries tight simultaneously; if either R2 or R5 were mis-implemented as strict, this fails.
TC-ATM-04	50,001 / 0 / 50,000 / Active / True	$P_{a,2}, P_{bal,2}$	balance = $T+1$	C2	**Approved**. Confirms R5 tolerant one unit past T.

ID	Inputs (*bal* / *used* / *amt* / *status* / *internal*)	Partition	Boundary	Combo	Expected output / Rationale
TC-ATM-05	20,000,000 / 0 / 20,000,000 / Active / True	$P_{a,2}$ (upper edge)	last reachable amount, balance $= T$	–	**Approved**. Balance is set exactly to T so the test is deterministically Approved, rather than merely "eligible"; also lands exactly on the daily-limit boundary (§4.12, kills M2 and M3).
TC-ATM-06	1,000,000 / 0 / 20,050,000 / Active / True	$P_{a,4}$	upper boundary, range side (+50,000)	–	**Rejected_InvalidAmount**. Over-range but still a multiple, isolating the range failure from the granularity failure at TC-ATM-07.
TC-ATM-07	1,000,000 / 0 / 50,001 / Active / True	$P_{a,3}$	interior granularity boundary	–	**Rejected_InvalidAmount**. In-range but not a multiple of 50,000.
TC-ATM-08	499,999 / 0 / 500,000 / Active / True	$P_{bal,1}$	balance $= T - 1$	–	**Rejected_InsufficientBalance**.
TC-ATM-09	500,000 / 0 / 500,000 / Active / True	$P_{bal,2}$	balance $= T$	–	**Approved**. Equality is the boundary, and it is on the valid side.
TC-ATM-10	1,000,000 / 19,499,999 / 500,000 / Active / True	$P_{dl,1}$	daily sum $= 20{,}000{,}000 - 1$	–	**Approved**.
TC-ATM-11	1,000,000 / 19,500,000 / 500,000 / Active / True	$P_{dl,1}$	daily sum $= 20{,}000{,}000$	–	**Approved** (equality is valid).
TC-ATM-12	1,000,000 / 19,500,001 / 500,000 / Active / True	$P_{dl,2}$	daily sum $= 20{,}000{,}000 + 1$	–	**Rejected_DailyLimit**.
TC-ATM-13	1,000,000 / 0 / 500,000 / Frozen / True	$P_{s,2}$	categorical status	–	**Rejected_InvalidAccount**, regardless of otherwise-perfect inputs (precedence R1 first).
TC-ATM-14	500,000 / 0 / 500,000 / Active / False	$P_{i,2}$	fee threshold shift	C5	**Rejected_InsufficientBalance** – identical balance/amount to TC-ATM-09, but fee now 50,000 moves T to 550,000 $>$ *balance*.
TC-ATM-15	500,000 / 19,500,000 / 500,000 / Active / True	– (R3,R5 both tight)	double derived boundary (balance $= T$ *and* daily sum $= 20{,}000{,}000$)	C3	**Approved** – both boundaries satisfied simultaneously.

ID	Inputs (*bal* / *used* / *amt* / *status* / *internal*)	Partition	Boundary	Combo	Expected output / Rationale
TC-ATM-16	499,999 / 19,500,001 / 500,000 / Active / True	– (R3,R5 both fail)	daily sum = 20,000,001 *and* balance $< T$ simultaneously	C4	**Rejected_DailyLimit** – both R3 and R5 fail; under the specified precedence (R3 before R5) the reported reason is DailyLimit. This is the only test able to distinguish the specified precedence from a reversed one (§4.12, M11).
TC-ATM-17	−100 / 0 / 500,000 / Active / True	invalid *balance* < 0 (deep invalid, sanity)	–	–	**Rejected_InsufficientBalance** – illustrates that the domain technically allows a negative balance (e.g. an already-overdrawn account) and the rule still resolves to a single, well-defined output.
TC-ATM-18	1,000,000 / 0 / 60,000 / Active / True	$P_{a,3}$	interior granularity boundary, chosen to be divisible by 10,000 but not 50,000	–	**Rejected_InvalidAmount**. Needed because TC-ATM-07's 50,001 fails divisibility under *any* plausible modulus mutant and therefore cannot distinguish the correct modulus from a wrong one (§4.12, M8).
TC-ATM-19	520,000 / 0 / 500,000 / Active / False	$P_{bal,1}$	balance strictly between the correct and an incorrect fee threshold	C6	**Rejected_InsufficientBalance**. Needed because TC-ATM-14's balance (500,000) sits below every plausible fee threshold and so cannot distinguish "fee removed" from "fee has the wrong magnitude" (§4.12, M6 vs. M7).
TC-ATM-20	1,000,000 / 0 / 0 / Active / True	$P_{a,1}$	amount = 0 (mandated deep-invalid boundary; also a multiple of 50,000)	C7	**Rejected_InvalidAmount**. The only test able to distinguish a correctly-enforced lower bound from one that has been removed or lowered to 0 (§4.12, M10).
TC-ATM-21	580,000 / 0 / 500,000 / Active / False	$P_{a,2}, P_{bal,2}, P_{i,2}$	fee threshold, External-Approved	C8	**Approved**. The suite's only Approved External test; balance sits strictly between the correct threshold (550,000) and an overcharging-fee threshold (600,000), so it is the unique test able to kill a mutant that charges too *high* a fee (§4.12, M13).

ID	Inputs (*bal* / *used* / *amt* / *status* / *internal*)	Partition	Boundary	Combo	Expected output / Rationale
TC-ATM-22	1,000,000 / 0 / 49,999 / Frozen / True	$P_{a,1}, P_{s,2}$	R1 *and* R2 simultaneously invalid	C9	**Rejected_InvalidAccount**. R1 and R2 both fail at once; under the specified precedence (R1 before R2) the reported reason is InvalidAccount. This is the only test able to distinguish the specified R1-before-R2 precedence from a reversed one (§4.12, M14).

Coverage check. Tables 10 and 11 read the suite in the opposite direction: starting from each partition (§4.6) and each boundary (§4.8–4.9), they name the tests that exercise it, so that a class or boundary without a test would be visible as an empty cell.

Table 10: ATM partition coverage: every class of §4.6 and a test that represents it

Partition ID	Mathematical definition	Valid / Invalid	Representative test
$P_{a,1}$	$\{x \in \mathbb{Z} \mid x < 50{,}000\}$	Invalid	TC-ATM-02
$P_{a,2}$	$\{x \in \mathbb{Z} \mid 50{,}000 \leq x \leq 20{,}000{,}000 \wedge x \equiv 0 \pmod{50{,}000}\}$	Valid	TC-ATM-01
$P_{a,3}$	$\{x \in \mathbb{Z} \mid 50{,}000 \leq x \leq 20{,}000{,}000 \wedge x \not\equiv 0 \pmod{50{,}000}\}$	Invalid	TC-ATM-07
$P_{a,4}$	$\{x \in \mathbb{Z} \mid x > 20{,}000{,}000\}$	Invalid	TC-ATM-06
$P_{bal,1}$	$\{x \in \mathbb{Z} \mid x < amount + fee\}$	Invalid	TC-ATM-08
$P_{bal,2}$	$\{x \in \mathbb{Z} \mid x \geq amount + fee\}$	Valid	TC-ATM-09
$P_{dl,1}$	$\{(d, a) \mid d + a \leq 20{,}000{,}000\}$	Valid	TC-ATM-11
$P_{dl,2}$	$\{(d, a) \mid d + a > 20{,}000{,}000\}$	Invalid	TC-ATM-12
$P_{s,1}$	{Active}	Valid	TC-ATM-01
$P_{s,2}$	{Frozen}	Invalid	TC-ATM-13
$P_{i,1}$	{True} (fee= 0)	Valid (numerically neutral)	TC-ATM-01
$P_{i,2}$	{False} (fee= 50,000)	Valid (numerically neutral)	TC-ATM-14

Every listed partition has at least one representative test in the suite (Table 9); completeness holds because every input value falls in exactly one listed class per input dimension, and the joint suite covers the Cartesian combinations identified in §4.9.

Table 11: ATM boundary coverage: every boundary of §4.8–4.9 and the tests that exercise it

Boundary ID	Definition	Type	Covered by	Expected change
B-amt-lo	$amount = 50{,}000$	Input-domain	TC-ATM-02, -03	Invalid→valid-range
B-amt-hi	$amount = 20{,}000{,}000$	Input-domain	TC-ATM-05, -06	Valid-range→invalid
B-amt-div	$amount \bmod 50{,}000 = 0$	Invalid/valid	TC-ATM-07	Multiple→non-multiple
B-bal	$balance = amount + fee$	Derived	TC-ATM-08, -09, C1/C2	Rejected→Approved
B-daily	$dailyUsed + amount = 20{,}000{,}000$	Derived	TC-ATM-10, -11, -12	Approved → Rejected_DailyLimit
B-status	Active ↔ Frozen	Logical	TC-ATM-13	Any → Rejected_InvalidAccount
B-fee	*isInternal* True↔False	Multi-variable interaction	TC-ATM-14, C5	Approved ↔ Rejected_InsufficientBalance
B-multi1	B-bal ∧ B-daily simultaneously	Multi-variable interaction	TC-ATM-15/16, C3/C4	Confirms precedence order (R3/R5)
B-fee-hi	$balance = amount + 100{,}000$ (overcharge threshold)	Multi-variable interaction	TC-ATM-21, C8	Isolates the External-Approved path; distinguishes correct fee from an overcharging one
B-multi2	R1 ∧ R2 simultaneously invalid	Multi-variable interaction	TC-ATM-22, C9	Confirms precedence order (R1/R2)
B-out	5-way D_{out} partition	Output	entire suite	Every rejection reason is individually reachable

4.11 Mutation Model

Table 12: ATM mutants

ID	Original	Mutated	Fault type	Targeted partition/boundary
M1	$balance \geq amount + fee$ (R5)	$balance > amount + fee$	Relational operator	Balance equality boundary (T)
M2	$dailyUsed + amount \leq 20{,}000{,}000$ (R3)	$dailyUsed + amount < 20{,}000{,}000$	Relational operator	Daily-limit equality boundary
M3	20,000,000 (upper amount bound, R2)	19,999,999	Boundary constant −1	Amount upper boundary

ID	Original	Mutated	Fault type	Targeted partition/boundary
M4	20,000,000 (upper amount bound, R2)	20,000,001	Boundary constant +1	Amount upper boundary – **equivalent mutant** (see discussion below)
M5	Gate requires *both* to hold (R1 ∧ R2)	Gate requires only *either* (R1 ∨ R2); R3/R5 unchanged	Logical connective (AND→OR)	Interaction of status and amount validity (only the R1/R2 gate is mutated; R3, R5 are unaffected)
M6	*fee* = 50,000 if External	*fee* = 0 always (fee removed)	Missing computation	Fee/isInternal interaction
M7	*fee* = 50,000 if External	*fee* = 5,000 if External	Wrong constant	Fee magnitude
M8	*amount* ≡ 0 (mod 50,000) (R2)	*amount* ≡ 0 (mod 10,000)	Wrong divisibility constant	Amount divisibility boundary
M9	*accountStatus* = Active (R1)	*accountStatus* = Frozen (condition reversed)	Reversed categorical condition	Status boundary
M10	50,000 (lower amount bound, R2)	0	Boundary constant (lower bound removed)	Amount lower boundary
M11	Precedence order R1, R2, R3, R5	Precedence order R1, R2, R5, R3 (R3/R5 swapped)	Control-flow reordering	Multi-failure precedence
M12	*dailyUsed* + *amount* ≤ 20,000,000 (R3)	*dailyUsed* ≤ 20,000,000 (amount dropped from sum)	Arithmetic mutation	Daily-limit derived boundary
M13	*fee* = 50,000 if External	*fee* = 100,000 if External	Wrong constant	Fee magnitude (overcharge direction)
M14	Precedence order R1, R2, R3, R5	Precedence order R2, R1, R3, R5 (R1/R2 swapped)	Control-flow reordering	Multi-failure precedence (status/amount)

4.12 Test–Mutant Analysis

Table 13 reports **analytically expected** killing: for each (test, mutant) pair, whether the mutant's specification-implied output for that test's input differs from the original specification's expected output. `K` means "this test's input makes the mutant compute a different result than the original," derived by hand; the matrix was afterwards re-computed by executing the mutants and matched cell for cell (§4.13).

Every cell below was obtained by substituting the test's five input values into both the original rule set (§4.2) and the mutated rule set, evaluating each of R1–R5 explicitly under the fixed precedence order, and comparing the two resulting elements of D_{out}; `K` is recorded only when the two elements differ. A cell is `-` when an earlier rule already fails identically in both the original and the mutant (the mutated rule is never reached, so it cannot possibly influence the output for that test).

Table 13: ATM Test×Mutant matrix (analytically expected; K=kill, S=survive (reached, no observable difference), –=mutated rule not reached)

Test	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10	M11	M12	M13	M14
TC-ATM-01	S	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-02	–	–	S	S	K	–	–	S	K	S	–	–	–	S
TC-ATM-03	K	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-04	S	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-05	K	K	K	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-06	–	–	S	S	K	–	–	S	K	S	–	–	–	S
TC-ATM-07	–	–	S	S	K	–	–	S	K	S	–	–	–	S
TC-ATM-08	S	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-09	K	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-10	S	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-11	S	K	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-12	–	S	S	S	S	–	–	S	K	S	S	K	–	S
TC-ATM-13	–	–	–	–	K	–	–	–	K	–	–	–	–	S
TC-ATM-14	S	S	S	S	S	K	S	S	K	S	S	S	S	S
TC-ATM-15	K	K	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-16	–	S	S	S	S	–	–	S	K	S	K	K	–	S
TC-ATM-17	S	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-18	–	–	S	S	K	–	–	K	K	S	–	–	–	S
TC-ATM-19	S	S	S	S	S	K	K	S	K	S	S	S	S	S
TC-ATM-20	–	–	S	S	K	–	–	S	K	K	–	–	–	S
TC-ATM-21	S	S	S	S	S	S	S	S	K	S	S	S	K	S
TC-ATM-22	–	–	–	–	S	–	–	–	K	–	–	–	–	K

Discussion of key entries. M1 (R5 strict) is killed exactly by the four tests that sit precisely on $balance = T$ (TC-ATM-03, -05, -09, -15); every other test that reaches R5 has $balance \neq T$ and so cannot distinguish $\geq$ from $>$, confirming that only an equality-exact test can kill a strict-relational mutant. M2 is killed exactly by the three tests with $dailyUsed + amount = 20{,}000{,}000$ precisely (TC-ATM-05, -11, -15); TC-ATM-16, whose sum is 20,000,001 and therefore already off the boundary on both sides, does *not* kill M2 – both the original ($\leq$) and the mutant ($<$) rules reject it identically, since neither reads the sum as exactly 20,000,000. M3 is killed only by TC-ATM-05 (the unique test with $amount = 20{,}000{,}000$ exactly); **M4 is an equivalent mutant** (§4.12 discussion below) and is killed by no test, correctly. M5 (the R1/R2 gate weakened from AND to OR) is killed whenever exactly one of R1, R2 is false while the other is true – TC-ATM-02, -06, -07, -18, -20 (amount invalid, status valid) and TC-ATM-13 (status invalid, amount valid); TC-ATM-13 is the most illustrative case, since under M5 the OR-gate lets a *frozen* account's withdrawal proceed to, and pass, R3/R5, so the mutant wrongly *approves* it, which is a qualitatively worse failure than the amount-invalid cases (a categorical precondition is silently bypassed, not merely an off-by-one on a numeric range) – and it is precisely because TC-ATM-13 tests the *complete* four-rule predicate on a Frozen account, rather than status in isolation, that it is able to expose this. M9 (reversed status condition) is killed by *every* test in the suite: on the nineteen Active-status tests the mutant instantly emits Rejected_InvalidAccount, which differs from each test's actual (non- InvalidAccount) expected output, while on the one Frozen test (TC-ATM-13) the mutant does the mirror-image wrong thing and approves it. This universal kill rate is an expected property of a coarse, single-bit categorical reversal on a field the suite deliberately keeps at its valid value almost everywhere, not evidence of a flawed matrix. M6 and M7 both mutate only the $isInternal$ = False branch of R4, so on every Internal test that reaches R4/R5, the fee rule is still executed but computes the identical value ($fee = 0$) under both the original and either mutant, and both columns are correctly S (reached, no observable difference) there – not –, since R4 is not skipped, only its unmutated branch is taken. TC-ATM-14 ($balance$ = 500,000, External) kills M6 (fee removed) but *survives* M7 (fee reduced

to 5,000), because 500,000 is below *both* the correct threshold (550,000) and the wrong one (505,000); only TC-ATM-19 (*balance* = 520,000, strictly between the two thresholds) kills M7. M8 (wrong divisibility modulus) is killed only by TC-ATM-18 (*amount* = 60,000, divisible by 10,000 but not 50,000); TC-ATM-07 (*amount* = 50,001) is divisible by neither modulus and is therefore rejected identically by the original and the mutant – a concrete instance of why a mutant-killing claim must be verified by substitution rather than assumed from the mutant's superficial description. M10 (lower bound removed) is killed only by TC-ATM-20 (*amount* = 0): every other amount below 50,000 present in the suite (TC-ATM-02, 49,999) is also non-divisible by 50,000 and so remains invalid even under the weakened lower bound, whereas 0 is itself a multiple of 50,000 and is therefore uniquely capable of exposing a missing or lowered floor. M11 (precedence swap) is killed only by TC-ATM-16, the sole test in which R3 *and* R5 fail simultaneously; TC-ATM-12 (R3 alone fails) and TC-ATM-08/-17 (R5 alone fails) all survive M11, because when only one of the two rules fails, checking it first or second yields the same reported reason either way (§4.9's discussion of C4 proves this algebraically for all four pass/fail combinations). M12 (amount dropped from the daily-limit sum) is killed by TC-ATM-12 and TC-ATM-16, both of which have *dailyUsed* alone below 20,000,000 while *dailyUsed* + *amount* exceeds it – the exact condition under which ignoring *amount* in the sum changes R3 from failing to (incorrectly) passing. M13 (fee overcharged to 100,000) is killed only by TC-ATM-21, the sole test with *balance* strictly between the correct External threshold (550,000) and the overcharging threshold (600,000); TC-ATM-14 and TC-ATM-19, the suite's other two External tests, both have *balance* below *both* thresholds and so are rejected identically under the original rule and M13, exactly mirroring why TC-ATM-14 alone could not distinguish M6 from M7. M14 (R1/R2 precedence swap) is killed only by TC-ATM-22, the sole test in which R1 *and* R2 fail simultaneously; every other test has at most one of R1, R2 false, and (by the same algebraic argument §4.9 gives for C4/M11) checking either rule first then yields the same reported reason, so M14 – structurally the same class of mutant as M11, but on the other adjacent rule pair – can only be exposed by a genuine double failure of that specific pair.

Of the fourteen mutants, M4 is the one **equivalent mutant**: because every valid amount is already a multiple of 50,000, and 50,000 does not divide 20,000,001, raising the upper bound from 20,000,000 to 20,000,001 admits no additional value into the valid amount set under any test, so the mutant computes the identical output as the original specification for every possible input, not merely for the tests in this suite. It is excluded from the non-equivalent-mutant denominator of the mutation-score definition in §2.5. All thirteen remaining mutants (M1–M3, M5–M14) are killed by at least one test in the twenty-two-test suite, so, restricted to those thirteen non-equivalent mutants, this analytically expected matrix supports full (13/13) expected coverage; this is a claim about the *design* of the suite against the stated specification (its executed confirmation and a tool-measured score on an independent mutant population are reported in §4.13), and (per Section 6) the mutant set was itself constructed with knowledge of the suite's boundaries, so this full coverage should not be read as unbiased evidence that boundary tests generally dominate interior ones.

4.13 Executed Validation

This subsection reports the executed stage of §3.7 for the ATM validator.

Implementation decisions. The specification leaves two points open, so the implementation makes an explicit choice for each and tests it. First, $dailyUsed < 0$, which §4.2 excludes from the domain, raises an argument-range error instead of returning an element of D_{out}; this check runs before R1, so a frozen account with a negative *dailyUsed* also raises it. Second, R3 is evaluated in the rearranged form $dailyUsed > 20{,}000{,}000 - amount$. It is equivalent to R3 for every input that reaches it (R2 has already bounded *amount*), but unlike the direct sum it cannot overflow

when *dailyUsed* is close to the largest representable integer; the specification does not address overflow.

Suite execution. All 87 ATM tests passed (0.48 s): the 22 designed tests TC-ATM-01–22, 18 supplementary boundary and guard tests, 39 matrix-reproduction checks and 8 properties (Table 1).

Matrix reproduction. All 14 columns of Table 13 were reproduced exactly: every `K` cell is reproduced, and no test kills a mutant that the matrix marks `S` or `-`. The executed score is therefore 13/13 non-equivalent mutants, as derived in §4.12. The equivalence of M4 was additionally checked by executing the original and M4 for every integer amount in $[-100{,}000,\ 20{,}200{,}000]$ (20,300,001 values, all other rules passing); no output differed. The reproduction depends on one reading of mutant semantics that Table 12 states only informally: for M5 ($\mathrm{R1} \wedge \mathrm{R2} \rightarrow \mathrm{R1} \vee \mathrm{R2}$), a single failure of R1 or R2 lets evaluation continue to R3 and R5, while a double failure is reported as Rejected_InvalidAccount (R1 first). This is the reading under which TC-ATM-22 survives M5, as Table 13 states.

Tool-generated mutants. Stryker.NET generated 23 mutants; one was ignored by the tool as a redundant block removal, and all other 22 were killed (0 survived; measured mutation score 100 %). By operator, the killed mutants are equality 10, conditional 4, arithmetic 3, logical 2, negation 2 and statement removal 1. The designed suite alone kills 21 of the 22. The one it does not kill removes the $dailyUsed \geq 0$ guard; that input lies outside the domain of §4.2, so by construction no designed test can reach it, and it is killed by the supplementary guard tests. Every tool-generated mutant inside the modelled domain, including the relational mutants on $balance < T$ and on the rearranged R3, is killed by the designed suite.

4.14 Case Study Discussion

The ATM case study demonstrates that even a specification built from five apparently independent business rules produces a joint valid region that is the *intersection*, not the union, of five per-variable conditions – and that two of the five rules (R3, R5) are defined over sums/derived thresholds rather than raw inputs. The single most important lesson is Figure 3: a categorical, seemingly unrelated input (*isInternal*) silently relocates a numeric boundary ($balance \geq T$) by a fixed offset, so no amount of independently maximal single-variable coverage of *isInternal* and *balance* would ever exercise the case where that relocation is exactly what determines the outcome.

5 Case Study II: Quadratic Equation Solver

5.1 Problem Context

Given integer coefficients a, b, c, classify the number and nature of the real solutions of $ax^2 + bx + c = 0$. Unlike the ATM case, the three inputs here are not coupled by separate policy rules but by a single algebraic identity – the discriminant – whose sign alone determines the qualitative output once the leading-coefficient degeneracy ($a = 0$) has been handled.

5.2 Formal Specification

$$Solve(a, b, c) \rightarrow D_{out}, \qquad a, b, c \in \mathbb{Z}.$$

Bounded domain (assumption, adopted to make systematic testing tractable).

$$D_a = D_b = D_c = \{x \in \mathbb{Z} \mid -100 \leq x \leq 100\}, \qquad D = D_a \times D_b \times D_c.$$

This bound is adopted purely to make BVA over a, b, c's own domain edges well-defined (Section 5.8); the classification logic itself (below) does not depend on this bound and would apply unchanged for any $a, b, c \in \mathbb{Z}$.

Case A — $a \neq 0$ (quadratic). Define $\Delta = b^2 - 4ac$.

$$\Delta < 0 \Rightarrow \text{NoRealRoots} \tag{15}$$
$$\Delta = 0 \Rightarrow \text{OneRepeatedRealRoot} \tag{16}$$
$$\Delta > 0 \Rightarrow \text{TwoDistinctRealRoots} \tag{17}$$

Case B — $a = 0$ (degenerates to linear $bx + c = 0$).

$$b \neq 0 \Rightarrow \text{OneLinearRoot} \tag{18}$$
$$b = 0,\ c \neq 0 \Rightarrow \text{NoSolution} \tag{19}$$
$$b = 0,\ c = 0 \Rightarrow \text{InfinitelyManySolutions} \tag{20}$$

Output domain.

$$D_{out} = \{\text{NoRealRoots},\ \text{OneRepeatedRealRoot},\ \text{TwoDistinctRealRoots},\ \text{OneLinearRoot},\ \text{NoSolution},\ \text{InfinitelyManySolutions}\}$$

Assumption. The specification asks only for the real-root *classification*, not the numeric roots themselves, and does not ask the solver to distinguish complex-conjugate roots by value; "no real roots" covers the entire $\Delta < 0$ region without further subdivision, since the problem statement gives no requirement to report complex root values.

5.3 Input and Output Domains

Table 14: Quadratic solver domains

Input	Domain	Note
a	$\{-100, \ldots, 100\} \subset \mathbb{Z}$	Leading coefficient; $a = 0$ is the case-split boundary
b	$\{-100, \ldots, 100\} \subset \mathbb{Z}$	Linear coefficient
c	$\{-100, \ldots, 100\} \subset \mathbb{Z}$	Constant term

The output domain's six labels are mutually exclusive and, together with Cases A/B above, collectively exhaustive over all of D: every $(a, b, c) \in D$ falls into exactly one of $a \neq 0 \wedge \Delta \lesseqgtr 0 / = 0$ or $a = 0 \wedge (b \neq 0$ or $(b = 0 \wedge c \lesseqgtr 0 / = 0))$.

5.4 Constraints and Dependencies

a, b, c are not independently meaningful to the output: the output is a function of the single derived quantity $\Delta = b^2 - 4ac$ when $a \neq 0$, and of a nested categorical case split on b, c when $a = 0$. Formally,

$$\mathit{Solve}(a, b, c) = g(\mathbf{1}[a = 0],\ b,\ c,\ \Delta(a, b, c))$$

i.e. the output factors through Δ (and the $a = 0/b = 0/c = 0$ indicators), not through a, b, c individually – exactly analogous to the ATM case's factoring through *fee* and *dailyUsed+amount*.

5.5 Representative and Nominal Configurations

A single nominal tuple cannot represent both the quadratic and linear behavioural regions (they are disjoint on $a = 0$ vs. $a \neq 0$), nor the three qualitatively different quadratic outcomes. It is also necessary, following §2.3, to keep two conceptually different kinds of representative value apart: a *nominal/interior* configuration sits safely inside a partition, away from every boundary of that partition, whereas a *canonical boundary* configuration is deliberately placed on, or immediately adjacent to, a behavioural boundary. $\Delta = 0$ and $(0, 0, 0)$ are boundary configurations, not nominal ones, and are labelled as such below rather than folded into a single undifferentiated "baseline" table.

Table 15: Nominal (interior) configurations

Configuration	a	b	c	Justification (why it is deep interior for its region)
Two-root nominal	1	7	6	$\Delta = 49 - 24 = 25 > 0$, far from $\Delta = 0$; factors as $(x + 1)(x + 6)$ – unremarkable two-root case
No-real-roots nominal	1	1	10	$\Delta = 1 - 40 = -39 < 0$, far from $\Delta = 0$ – unremarkable no-real-roots case
Linear nominal	0	2	-4	$a = 0$ is fixed here only because it is the case-defining value for this branch (Case B); $b = 2 \neq 0$ is chosen deep away from the nested $b = 0/c = 0$ boundaries inside that branch

Table 16: Canonical boundary configurations

Configuration	a	b	c	Justification (why it is deliberately on/near a boundary)
Repeated-root boundary	1	2	1	$\Delta = 4 - 4 = 0$ exactly – the central derived boundary itself, *not* an interior value
No-real-roots, boundary-adjacent	1	1	1	$\Delta = 1 - 4 = -3$, the nearest reachable negative discriminant (§5.8)
Two-distinct-roots, boundary-adjacent	1	3	2	$\Delta = 9 - 8 = 1$, the nearest reachable positive discriminant (§5.8)
No-solution boundary	0	0	5	$a = 0, b = 0, c \neq 0$: on the nested $b = 0$ boundary inside Case B
Degenerate multi-boundary	0	0	0	$a = 0 \wedge b = 0 \wedge c = 0$ simultaneously: three nested boundaries coincide at a single point

Each nominal configuration is "ordinary" for its region precisely because its Δ (or, for the linear branch, its b, c) sits far from every boundary of that region; each boundary configuration is ordinary in the opposite sense – it is chosen *because* it sits on or immediately next to a boundary, and must therefore never be described as "deep" or "interior." TC-QE-01 (§5.10) is built from the two-root nominal configuration; TC-QE-02, -03, -07, -05, -06 are built from the boundary configurations.

5.6 Equivalence Partitioning

a. $P_{a^-} = \{a \in D_a \mid a < 0\}$, $P_{a^0} = \{a \in D_a \mid a = 0\}$, $P_{a^+} = \{a \in D_a \mid a > 0\}$ – exhaustive, mutually exclusive.

Discriminant partitions (for $a \neq 0$, i.e. restricted to $P_{a^-} \cup P_{a^+}$).

$$P_{\Delta^-} = \{(a,b,c) \in D \mid a \neq 0,\ \Delta < 0\} \tag{21}$$
$$P_{\Delta^0} = \{(a,b,c) \in D \mid a \neq 0,\ \Delta = 0\} \tag{22}$$
$$P_{\Delta^+} = \{(a,b,c) \in D \mid a \neq 0,\ \Delta > 0\} \tag{23}$$

These three are exhaustive and mutually exclusive over $\{a \neq 0\}$ because $\Delta < 0, = 0, > 0$ exhaust $\mathbb{Z}$'s trichotomy.

Nested partitions for $a = 0$ (linear degeneration).

$$P_{b\neq 0} = \{(0,b,c) \in D \mid b \neq 0\} \qquad \Rightarrow \text{OneLinearRoot} \tag{24}$$
$$P_{b0,c\neq 0} = \{(0,0,c) \in D \mid c \neq 0\} \qquad \Rightarrow \text{NoSolution} \tag{25}$$
$$P_{b0,c0} = \{(0,0,0)\} \qquad \Rightarrow \text{InfinitelyManySolutions} \tag{26}$$

$P_{b\neq 0} \cup P_{b0,c\neq 0} \cup P_{b0,c0} = \{0\} \times D_b \times D_c$ exactly, with the three classes pairwise disjoint by construction (first split on b, then on c) – this is a genuinely *nested* partition, not a flat one, and must be represented as such (a flat partition of $a = 0$ into "$b = 0$" / "$b \neq 0$" would miss the further c-split that distinguishes NoSolution from InfinitelyManySolutions).

Joint valid region. Every $(a,b,c) \in D$ is valid input (there is no "invalid" input class here – any integer triple is a legitimate query to the classifier); D itself is partitioned, not filtered, by $P_{a^0} \cup P_{a^-} \cup P_{a^+}$, each further sub-partitioned as above.

5.7 Partition Diagrams

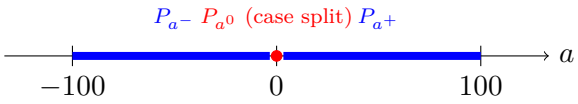


Figure 4: One-dimensional partition of a. $a = 0$ is a single, isolated point that is not merely a boundary between two numeric ranges but the switch between two entirely different classification algorithms (quadratic vs. linear).

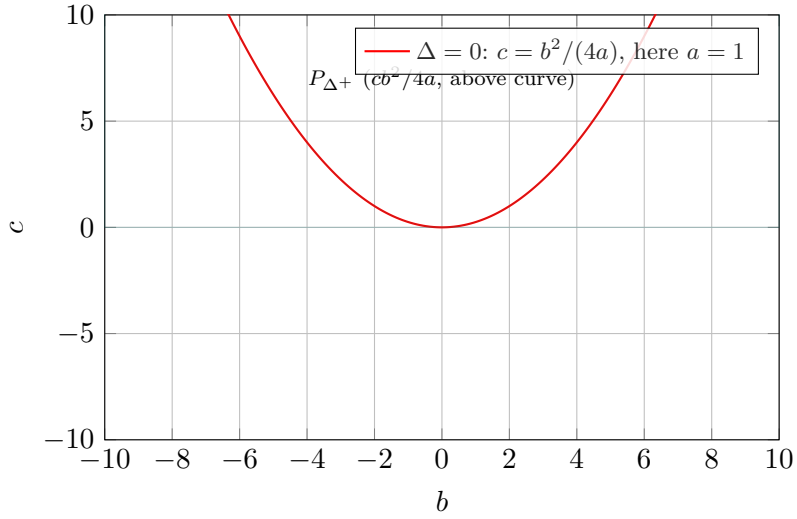


Figure 5: Conceptual (b, c) slice at fixed $a = 1$, showing the discriminant curve $b^2 - 4c = 0$ (equivalently $c = b^2/4$). This is a 2D *slice* of the 3D (a, b, c) space with a held fixed at 1 – it is not an exact picture of the full discriminant surface $b^2 - 4ac = 0$ in 3D, only one cross-section of it; the curve's shape (and even its convexity direction) changes with the sign and magnitude of a.

5.8 Boundary Value Analysis

Important derived boundary: $\Delta = 0$. *This is a joint boundary in (a, b, c)-space, not a boundary of b or c alone.* The surface $b^2 - 4ac = 0$ is two-dimensional inside the 3-dimensional domain D; "crossing" it can be achieved by moving any one of a, b, or c while holding the other two fixed, and each such move constitutes a legitimate boundary test.

Discrete-domain reachability of Δ near zero. Because $a, b, c \in \mathbb{Z}$: for any integer b, $b^2 \equiv 0$ or $1 \pmod 4$ (even b: $b^2 \equiv 0$; odd b: $b^2 \equiv 1$); and $4ac \equiv 0 \pmod 4$ always. Therefore

$$\Delta = b^2 - 4ac \equiv 0 \text{ or } 1 \pmod 4.$$

Consequently $\Delta = -1$ and $\Delta = -2$ are **not attainable** for any integer a, b, c – a naive "test $\Delta = -1, 0, +1$" BVA rule would silently select an impossible input and produce a vacuous, non-executable test case. The nearest reachable values below and above zero must instead be identified by construction:

Table 17: Reachable discriminant values near the boundary, with witnesses

Target Δ	Witness (a, b, c)	Check
-3 (nearest reachable negative)	$(1, 1, 1)$	$1 - 4 = -3$
-4 (also reachable)	$(1, 0, 1)$	$0 - 4 = -4$
0 (boundary)	$(1, 2, 1)$	$4 - 4 = 0$
1 (nearest reachable positive)	$(1, 3, 2)$	$9 - 8 = 1$
4 (also reachable, $\equiv 0$)	$(1, 4, 3)$	$16 - 12 = 4$

The witnesses above establish that -3, 0, and 1 are all reachable, but the crossing sequence used in §5.9 to demonstrate the qualitative transition (Q7) instead uses $(1, 1, 1) \to (1, 2, 1) \to (1, 3, 1)$, i.e. $a = 1$ and $c = 1$ held fixed and *only* b incremented ($1 \to 2 \to 3$): this gives discriminants $-3 \to 0 \to 5$. Both -3 and 5 are reachable ($-3 \equiv 1$, $5 \equiv 1 \pmod 4$), and the sequence is a genuinely single-variable crossing, unlike a sequence that changes c partway

through, which would conflate a b-driven and a c-driven step and no longer isolate the effect of a single coefficient.

Coefficient domain boundaries. The domain extremes and case-split neighbours $a, b, c \in \{-100, -1, 0, 1, 100\}$ are the candidate *direct* boundaries of D_a, D_b, D_c identified by Stage 3 of the methodology (§3.3) – but these do not all carry equal behavioural weight, and only the behaviourally significant subset (Table 18 below) is retained in the final suite (§5.10); a domain-extreme value with no distinct branch of its own is not worth a dedicated test. $a = -100$ or $a = 100$ tests only that the classifier accepts the domain's extreme leading coefficients (no distinct branch is taken there beyond ordinary $a \neq 0$ processing); $a = 0$ and $a = 1$ (its nearest non-zero neighbour) are far more significant because they straddle the *behavioural* (case-split) boundary, not merely a domain-range boundary. $b = 0$ and $c = 0$ are significant only in conjunction with $a = 0$ (they gate the nested linear/no-solution/infinite-solutions split); when $a \neq 0$, $b = 0$ or $c = 0$ individually carry no special status beyond their ordinary contribution to Δ.

Table 18: Direct coefficient boundaries vs. their behavioural significance

Value	Domain-edge significance	Behavioural significance
$a = -100, 100$	Domain extremes	None beyond ordinary $a \neq 0$ path
$a = -1, 1$	Nearest neighbours of $a = 0$	High – adjacent to the case-split
$a = 0$	Interior of D_a	Maximal – switches algorithm entirely
$b = 0$ (with $a \neq 0$)	Interior of D_b	Low – ordinary Δ contribution
$b = 0$ (with $a = 0$)	Interior of D_b	Maximal – gates NoSolution vs. InfinitelyMany
$c = 0$ (with $a \neq 0$)	Interior of D_c	Low – ordinary Δ contribution
$c = 0$ (with $a = 0, b = 0$)	Interior of D_c	Maximal – selects InfinitelyManySolutions

5.9 Boundary Combinations

Table 19: Mandatory quadratic boundary combinations

Combination	Tuple	Why it matters
Q1	$(0, 0, 0)$	Deepest degenerate point: all three coefficients zero simultaneously.
Q2	$(0, 0, 5)$	$a = 0, b = 0, c \neq 0$: isolates NoSolution from InfinitelyManySolutions, which differ only in c.
Q3	$(0, 3, -1)$	$a = 0, b \neq 0$: linear case, c irrelevant to branch selection once $b \neq 0$.
Q4	$(1, 1, 1)$	$a \neq 0, \Delta < 0$ (reachable, $\Delta = -3$).
Q5	$(1, 2, 1)$	$a \neq 0, \Delta = 0$ (repeated root, boundary).
Q6	$(1, 3, 2)$	$a \neq 0, \Delta > 0$ (reachable, $\Delta = 1$).
Q7	$(1, 1, 1) \to (1, 2, 1) \to (1, 3, 1)$	Crossing tuple: holding $a = 1$ and $c = 1$ fixed and increasing only b ($1 \to 2 \to 3$) drives Δ through $-3 \to 0 \to 5$, i.e. through NoRealRoots $\to$ OneRepeatedRealRoot $\to$ TwoDistinctRealRoots under a genuinely single-variable (b-only) change – the qualitative-change demonstration required by §5.8/§5.14.2.
Q8	$(2, 4, 2)$ vs. $(1, 2, 1)$	Same $\Delta = 0$ boundary reached via a *different* coefficient combination (both a and c doubled): shows $\Delta = 0$ is a genuinely joint condition, not tied to one specific (a, b, c) triple.
Q9	$(-1, 0, 1)$	$a < 0$ combined with $\Delta = 0 - 4(-1)(1) = 4 > 0$: confirms sign of a does not change the discriminant classification logic itself.

Single-variable testing is insufficient here in the strongest possible sense: holding a and c at any fixed nominal value and only varying b across its own domain boundary values $(-100, -1, 0, 1, 100)$ will, for almost all fixed (a, c), never once land on $\Delta = 0$, because $\Delta = 0$ is a one-dimensional curve inside a 2D (b, c) slice (for fixed a), and a domain-boundary value of b chosen without reference to a, c has no reason to be one of the (at most two) roots of $b^2 = 4ac$. Only a boundary-combination test that solves for one variable in terms of the other two (as Q5, Q7, Q8 do) can reliably hit the derived boundary.

5.10 Test Suite Construction

Candidates were generated from: (i) the a-partition (3 classes) and nested $a = 0$ partitions (3 sub-classes); (ii) the Δ-partition (3 classes); (iii) the domain-edge boundaries of a, b, c (Table 18, §5.8, filtered to those with real behavioural significance – not every coefficient boundary is equally meaningful); (iv) the nine combinations Q1–Q9; (v) TC-QE-16, added after §5.5 distinguished nominal from boundary configurations and exposed that no existing test represented a genuinely deep-interior P_{Δ^-} point. Tests that would duplicate coverage already achieved (e.g. a plain $a = 100, b = 1, c = 1$ representative of P_{a^+} that adds nothing beyond what Q6-style tests already cover) were dropped. One deliberate exception is TC-QE-08 (Q7): its first two sub-tuples, $(1, 1, 1)$

and $(1, 2, 1)$, are numerically identical to TC-QE-03 and TC-QE-02 respectively, but TC-QE-08 is retained as a distinct test because it evaluates them as a single connected *sequence* (§5.9's crossing argument requires that all three points be reached by incrementing b alone, from a fixed starting tuple, not as independent, individually-chosen samples); no other retained test overlaps another's coefficients. The resulting sixteen tests are otherwise non-redundant, and are the complete quadratic suite; the same sixteen tests, in are listed below; the coverage of every partition and every boundary by these tests is checked directly after the table.

Table 20: Quadratic complete test suite, with full traceability

ID	(a, b, c)	Partition	Boundary	Combo	Expected output / Rationale
TC-QE-01	$(1, 7, 6)$	P_{Δ^+}	–	–	**TwoDistinctRealRoots** ($\Delta = 25$; two-root nominal/interior configuration, far from $\Delta = 0$).
TC-QE-02	$(1, 2, 1)$	P_{Δ^0}	$\Delta = 0$	Q5	**OneRepeatedRealRoot** ($\Delta = 0$ exactly; canonical repeated-root *boundary* configuration, not nominal).
TC-QE-03	$(1, 1, 1)$	P_{Δ^-}	nearest reachable $\Delta = -3$	Q4	**NoRealRoots** (boundary-adjacent configuration).
TC-QE-04	$(0, 2, -4)$	$P_{a^0} \cap P_{b\neq 0}$	–	Q3	**OneLinearRoot** (linear nominal/interior configuration, away from the nested $b = 0/c = 0$ boundaries).
TC-QE-05	$(0, 0, 5)$	$P_{b0,c\neq 0}$	$b = 0, c \neq 0$	Q2	**NoSolution** (boundary configuration).
TC-QE-06	$(0, 0, 0)$	$P_{b0,c0}$	$a = b = c = 0$	Q1	**InfinitelyManySolutions** (degenerate multi-boundary configuration, not nominal).
TC-QE-07	$(1, 3, 2)$	P_{Δ^+}	nearest reachable $\Delta = 1$	Q6	**TwoDistinctRealRoots** (boundary-adjacent configuration; distinct from the deep-interior TC-QE-01).
TC-QE-08	$(1, 1, 1) \to$ $(1, 2, 1) \to$ $(1, 3, 1)$	crosses $P_{\Delta^-} \to$ $P_{\Delta^0} \to P_{\Delta^+}$	full crossing	Q7	**NoRealRoots** $\to$ **OneRepeatedRealRoot** $\to$ **TwoDistinctRealRoots** ($\Delta = -3 \to 0 \to 5$); demonstrates a purely quantitative, single-variable b-change causing a qualitative output change.
TC-QE-09	$(2, 4, 2)$	P_{Δ^0}	$\Delta = 0$ via different triple	Q8	**OneRepeatedRealRoot** ($\Delta = 16 - 16 = 0$) – confirms $\Delta = 0$ is not tied to one specific (a, b, c).
TC-QE-10	$(-1, 0, 1)$	$P_{a^-} \cap P_{\Delta^+}$	a sign boundary	Q9	**TwoDistinctRealRoots** ($\Delta = 0 - 4(-1)(1) = 4 > 0$, regardless of a's sign).
TC-QE-11	$(100, 1, 1)$	P_{a^+} domain edge	$a = 100$	–	**NoRealRoots** ($\Delta = 1 - 400 = -399 < 0$); exercises the extreme leading coefficient.
TC-QE-12	$(-100, 1, 1)$	P_{a^-} domain edge	$a = -100$	–	**TwoDistinctRealRoots** ($\Delta = 1 + 400 = 401 > 0$).
TC-QE-13	$(1, 100, 1)$	P_{Δ^+}, b domain edge	$b = 100$	–	**TwoDistinctRealRoots** ($\Delta = 10{,}000 - 4 = 9{,}996 > 0$).

ID	(a, b, c)	Partition	Boundary	Combo	Expected output / Rationale
TC-QE-14	$(0, 100, 0)$	$P_{b\neq 0}$, b domain edge, $c = 0$	$a = 0, c = 0, b \neq 0$	–	**OneLinearRoot** ($x = 0$); shows $c = 0$ has no special status once $b \neq 0$ under $a = 0$.
TC-QE-15	$(1, 0, 1)$	P_{Δ^-}, $b = 0$ with $a \neq 0$	reachable $\Delta = -4$	–	**NoRealRoots**; contrasts with TC-QE-05 to show $b = 0$ means something entirely different when $a \neq 0$ versus $a = 0$.
TC-QE-16	$(1, 1, 10)$	P_{Δ^-}	–	–	**NoRealRoots** ($\Delta = 1 - 40 = -39$; the no-real-roots nominal/interior configuration, added so that the suite contains at least one Δ-negative representative that is genuinely deep interior rather than boundary-adjacent, per §5.5).

Coverage check. Tables 21 and 22 read the suite in the opposite direction: starting from each partition (§5.6) and each boundary (§5.8–5.9), they name the tests that exercise it.

Table 21: Quadratic partition coverage: every class of §5.6 and a test that represents it

Partition ID	Mathematical definition	Valid / Invalid	Representative test
P_{a^-}	$\{a \in D_a \mid a < 0\}$	Valid (all inputs valid)	TC-QE-10
P_{a^0}	$\{a \in D_a \mid a = 0\}$	Valid	TC-QE-04, -05, -06
P_{a^+}	$\{a \in D_a \mid a > 0\}$	Valid	TC-QE-01
P_{Δ^-}	$\{(a, b, c) \in D \mid a \neq 0, \Delta < 0\}$	Valid	TC-QE-03
P_{Δ^0}	$\{(a, b, c) \in D \mid a \neq 0, \Delta = 0\}$	Valid	TC-QE-02
P_{Δ^+}	$\{(a, b, c) \in D \mid a \neq 0, \Delta > 0\}$	Valid	TC-QE-01
$P_{b\neq 0}$	$\{(0, b, c) \in D \mid b \neq 0\}$	Valid	TC-QE-04
$P_{b0,c\neq 0}$	$\{(0, 0, c) \in D \mid c \neq 0\}$	Valid	TC-QE-05
$P_{b0,c0}$	$\{(0, 0, 0)\}$	Valid	TC-QE-06

Since every $(a, b, c) \in D$ is legitimate input to a mathematical classifier, all listed partitions are "valid" in the invalid-input sense (there is no malformed-input class); the valid/invalid column instead confirms the partitioning is exhaustive over D with no gap.

Table 22: Quadratic boundary coverage: every boundary of §5.8–5.9 and the tests that exercise it

Boundary ID	Definition	Type	Covered by	Expected change
B-a0	$a = 0$	Logical (case split)	TC-QE-04,-05,-06 vs. TC-QE-01,-02,-03	Quadratic algorithm $\rightarrow$ linear algorithm

Boundary ID	Definition	Type	Covered by	Expected change
B-delta	$\Delta = b^2 - 4ac = 0$	Derived	TC-QE-02, -08 (middle), -09	NoRealRoots/ TwoDistinct $\rightarrow$ OneRepeated
B-b0-nested	$b = 0$ (given $a = 0$)	Multi-variable interaction (nested)	TC-QE-05 vs. TC-QE-04	OneLinearRoot $\rightarrow$ {NoSolution, InfiniteMany}
B-c0-nested	$c = 0$ (given $a = 0, b = 0$)	Multi-variable interaction (nested)	TC-QE-06 vs. TC-QE-05	NoSolution $\rightarrow$ InfinitelyManySolutions
B-adom	$a \in \{-100, 100\}$	Input-domain	TC-QE-11, -12	No branch change; extreme-magnitude sanity
B-bdom	$b = 100$	Input-domain	TC-QE-13, -14	No branch change (quadratic branch); gates branch only under $a = 0$
B-reach	Nearest reachable Δ to 0 ($-3, +1$ vs. unreachable $-1, -2$)	Derived (discrete-domain)	TC-QE-03, -07	Confirms BVA neighbour selection is reachability-checked
B-cross	Δ crossing $-3 \rightarrow 0 \rightarrow 5$ via b alone ($a = 1, c = 1$ fixed)	Multi-variable interaction	TC-QE-08 (Q7)	Full qualitative-change demonstration

5.11 Mutation Model

Table 23: Quadratic solver mutants

ID	Original	Mutated	Fault type	Targeted partition/boundary
N1	$\Delta = b^2 - 4ac$	$\Delta = b^2 - 2ac$	Arithmetic (wrong constant)	Discriminant formula, all Δ classes
N2	$\Delta > 0 \Rightarrow$ Two roots	$\Delta \geq 0 \Rightarrow$ Two roots	Relational operator	$\Delta = 0$ boundary
N3	$\Delta < 0 \Rightarrow$ No roots	$\Delta \leq 0 \Rightarrow$ No roots	Relational operator	$\Delta = 0$ boundary
N4	$a = 0 \Rightarrow$ linear case	$a \neq 0 \Rightarrow$ linear case (condition reversed)	Reversed categorical condition	$a = 0$ boundary
N5	$\Delta < 0 \Rightarrow$ NoRealRoots, $\Delta > 0 \Rightarrow$ TwoDistinctRealRoots	labels swapped ($\Delta < 0 \Rightarrow$Two, $\Delta > 0 \Rightarrow$No)	Classification swap	Both Δ sign classes
N6	b^2	$2b$	Arithmetic (wrong expression)	Discriminant formula
N7	$b = 0, c = 0 \Rightarrow$ InfinitelyManySolutions	$b = 0, c = 0 \Rightarrow$ NoSolution	Wrong classification	Degenerate linear boundary
N8	$b = 0, c \neq 0 \Rightarrow$ NoSolution	$b = 0, c \neq 0 \Rightarrow$ InfinitelyManySolutions	Wrong classification	$b = 0$ nested boundary
N9	$b \neq 0 \Rightarrow$ OneLinearRoot (under $a = 0$)	$b \neq 0 \Rightarrow$ NoSolution	Wrong classification	$a = 0, b \neq 0$ boundary
N10	$4ac$	$4bc$	Arithmetic (wrong variable)	Discriminant formula

5.12 Test–Mutant Analysis

Every cell below was obtained by substituting the test's coefficients into both the original classification rule and the mutated one, computing each one's discriminant (or branch) *and* the resulting element of D_{out}, and comparing the two output elements – never the raw numeric discriminants. This distinction matters: as §2.5 stresses, a mutant that changes the numeric value of Δ without changing which side of a boundary it falls on (e.g. $\Delta = 1 \to 13$, both > 0) must be recorded as `S`, even though the number visibly changed.

Table 24: Quadratic Test×Mutant matrix (analytically expected; `K`=kill, `S`=survive, `-`=mutated rule not reached for this test)

Test	N1	N2	N3	N4	N5	N6	N7	N8	N9	N10
TC-QE-01	S	S	S	K	K	K	–	–	–	K
TC-QE-02	K	K	K	K	S	S	–	–	–	K
TC-QE-03	S	S	S	K	K	S	–	–	–	S
TC-QE-04	–	–	–	K	–	–	–	–	K	–
TC-QE-05	–	–	–	K	–	–	–	K	–	–
TC-QE-06	–	–	–	K	–	–	K	–	–	–
TC-QE-07	S	S	S	K	K	K	–	–	–	K
TC-QE-08	K	K	K	K	K	S	–	–	–	K
TC-QE-09	K	K	K	K	S	K	–	–	–	K
TC-QE-10	S	S	S	K	K	S	–	–	–	K
TC-QE-11	S	S	S	K	K	S	–	–	–	S
TC-QE-12	S	S	S	K	K	S	–	–	–	K
TC-QE-13	S	S	S	K	K	S	–	–	–	S
TC-QE-14	–	–	–	K	–	–	–	–	K	–
TC-QE-15	S	S	S	K	K	S	–	–	–	K
TC-QE-16	S	S	S	K	K	S	–	–	–	S

Discussion of key entries. N2 and N3 (the two relational-operator mutants at the $\Delta = 0$ boundary) are both killed exactly by TC-QE-02, TC-QE-08 (via its middle sub-case), and TC-QE-09 – the only three tests with $\Delta = 0$ exactly – and survive on every other test regardless of Δ's sign or magnitude, confirming that no interior-Δ test can ever kill either mutant. **N1 ($b^2 - 4ac \to b^2 - 2ac$) was recomputed by direct substitution for all sixteen tests rather than assumed**: N1 is killed only by TC-QE-02, TC-QE-08, and TC-QE-09 – precisely the three $\Delta = 0$ tests, because halving the ac term moves the discriminant away from exactly zero (e.g. at TC-QE-02, $\Delta = 4 - 4 = 0$ becomes $\Delta' = 4 - 2 = 2 > 0$, a classification change) while at every other test the sign of Δ happens to be preserved despite the numeric value changing (e.g. at TC-QE-01, $\Delta = 25 \to \Delta' = 37$, both > 0 – survives; at TC-QE-11, $\Delta = -399 \to \Delta' = -199$, both < 0 – survives). This is exactly the principle in §5.12's opening paragraph: a changed number only kills a mutant if the classification changes, and N1 is a much weaker mutant than a superficial reading of its formula would suggest. N6 ($b^2 \to 2b$) shows the same pattern even more starkly, surviving nine of its twelve applicable tests (including, by direct coincidence, TC-QE-02 and TC-QE-08's first two sub-cases, where $2b$ happens to equal $4ac$ or preserve the sign of $b^2 - 4ac$) and killed only where the arithmetic error happens to cross the zero boundary (TC-QE-01, -07, -09). N10 ($4ac \to 4bc$) survives four tests for two structurally different reasons, which must not be conflated: at TC-QE-03 $(1, 1, 1)$ and TC-QE-16 $(1, 1, 10)$, $a = b = 1$ exactly, so $ac = bc$ identically and the mutation has *no numeric effect at all* there ($\Delta' = \Delta$); at TC-QE-11 $(100, 1, 1)$ and TC-QE-13 $(1, 100, 1)$, by contrast, $ac \neq bc$ ($100 \neq 1$ and $1 \neq 100$ respectively) and Δ genuinely changes ($-399 \to -3$ and $9{,}996 \to 9{,}600$), but the mutant *survives anyway* because the sign of Δ happens to be preserved across that change – the same numeric-change-without-classification-change pattern already noted for N1 and N6, not a special property of $a = b$. N10 kills the remaining eight tests. **N4 (the $a = 0 / a \neq 0$ branch-selection condition reversed) is killed by every one of the sixteen tests**: reversing which branch handles which value of a makes the mutant apply the linear Case-B rules to every $a \neq 0$ input (ignoring Δ entirely) and the quadratic Case-A rules, with $\Delta = b^2$, to every $a = 0$ input, and no test in the suite happens to produce the same output under both algorithms – this is the expected signature of a mutant that swaps two entire branches of a case split rather than perturbing a single constant. N5 (sign classification swapped) is killed by every $\Delta \neq 0$ quadratic test (ten of twelve) and survives only the two tests that sit exactly at $\Delta = 0$ (TC-QE-02, -09), since N5's swap is defined only for the strict sign branches. N7, N8, N9 are each narrowly targeted at one specific nested sub-case of

the $a = 0$ branch and are accordingly killed by exactly the one or two tests that exercise that sub-case (TC-QE-06; TC-QE-05; TC-QE-04 and TC-QE-14, respectively), with every other test correctly marked – since the mutated code path is never reached for them.

5.13 Executed Validation

This subsection reports the executed stage of §3.7 for the quadratic solver.

Implementation decisions. Coefficients outside $[-100, 100]$ (§5.2) raise an argument-range error instead of returning an element of D_{out}. For $a \neq 0$, the sign of Δ is classified by testing $\Delta < 0$ first, then $\Delta > 0$, and returning OneRepeatedRealRoot otherwise. Table 23 states N2 and N3 only informally; this order is the reading under which both relational mutants misclassify exactly the $\Delta = 0$ tests, as §5.12 derives.

Suite execution. All 79 quadratic tests passed (0.55 s, including the exhaustive sweep below): 17 designed-suite tests (TC-QE-08 as two sequence checks), 26 supplementary boundary and guard tests, 28 matrix-reproduction checks, 6 properties and 2 exhaustive checks (Table 1).

Matrix reproduction. All 10 columns of Table 24 were reproduced exactly, giving an executed score of 10/10, as derived in §5.12.

Tool-generated mutants. Stryker.NET generated 24 mutants; three were ignored by the tool as redundant block removals, and all other 21 were killed (0 survived; measured mutation score 100 %). By operator, the killed mutants are equality 7, statement removal 6, arithmetic 4, conditional 2 and negation 2. The designed suite alone kills 15 of the 21. The six it does not kill each remove one of the six coefficient-range guards; those inputs lie outside the domain of §5.2, so by construction no designed test can reach them, and they are killed by the supplementary guard tests. The two $\Delta = 0$ relational mutants ($\Delta < 0 \to \Delta \leq 0$ and $\Delta > 0 \to \Delta \geq 0$, the tool-generated counterparts of N3 and N2) were killed by the designed-suite tests and by the matrix-reproduction checks, consistent with §5.12.

The run also exposed a practical hazard. In an earlier version of the implementation, the sign of Δ was classified with a relational pattern match (`< 0`, `> 0`, default). Stryker's boundary mutants of that form (`<= 0`, `>= 0`) make the default arm unreachable, which the C# compiler rejects, so four mutants were reported as compile errors and excluded from the score. The score was still reported as 100 %, yet it no longer covered the $\Delta = 0$ boundary at all. With the equivalent if-statement form, those four mutants compile and are killed. A measured mutation score is therefore only as informative as the set of mutants that the tool manages to compile; the excluded mutants have to be inspected, not only the score.

Exhaustive execution of the domain. Because D is finite, the classifier was executed on all $201^3 = 8{,}120{,}601$ triples. The check confirms that it never fails, that it agrees with a restatement of the specification in arbitrary-precision arithmetic, and that $\Delta \bmod 4 \in \{0, 1\}$ for every triple (§5.8). Table 25 gives the resulting distribution.

Table 25: Output classes over the full domain $[-100, 100]^3$

Output	Triples	Share
TwoDistinctRealRoots	5,055,984	62.26 %
NoRealRoots	3,023,112	37.23 %
OneLinearRoot	40,200	0.50 %
OneRepeatedRealRoot	1,104	0.0136 %
NoSolution	200	0.0025 %
InfinitelyManySolutions	1	0.00001 %
Total	8,120,601	100 %

Two claims made earlier in this case study can now be stated quantitatively. First, the reachability argument of §5.8 holds over the whole domain: among the 8,080,200 quadratic triples ($a \neq 0$), discriminants in $[-4, 4]$ occur only as -4 (338 triples), -3 (252), 0 (1,104), 1 (1,880) and 4 (1,754); $\Delta \in \{-2, -1, 2, 3\}$ never occurs. Second, the derived boundary $\Delta = 0$ occupies 1,104 of those 8,080,200 triples (0.0137 %). A test selected uniformly at random from that set hits it with probability 1.37×10^{-4}, so a random suite of 1,000 such tests contains no $\Delta = 0$ case with probability ≈ 0.87, and would then kill neither N2 nor N3, which differ from the original only at $\Delta = 0$. This is the quantitative form of the argument in §5.9 that the derived boundary has to be constructed (Q5, Q7, Q8), not sampled.

5.14 Case Study Discussion

The quadratic case study demonstrates a second, distinct flavour of multi-variable boundary: where the ATM's boundaries were sums and simple thresholds, the quadratic's central boundary ($\Delta = 0$) is a genuinely non-linear surface in 3-space, and its discreteness (Section 5.8) shows that BVA's usual "± 1" heuristic must be checked for reachability before being applied, not assumed. The nested $a = 0$ nested partition (linear / no-solution / infinite-solutions) further shows that a single categorical boundary ($a = 0$) can itself contain a second, hidden equivalence structure that only becomes visible once the outer boundary has already been crossed.

5.14.1 Comparison with Case Study I

Domain Comparison. Both case studies have multi-variable, mixed or purely-integer input domains. The ATM domain is unbounded ($\mathbb{Z}$ for the numeric inputs) plus two small categorical domains; the quadratic domain is a bounded cube $[-100, 100]^3 \subset \mathbb{Z}^3$ adopted purely as a testing convenience. The ATM's output domain is a flat 5-way categorical partition; the quadratic's is a flat 6-way categorical partition, but internally structured by a 2-level case split (Case A/B, then a further 3-way split inside each).

Partition Comparison. The ATM's partitions are almost all defined by *linear inequalities and a modulus condition* on single variables or simple sums. The quadratic's partitions are defined by a *quadratic form* ($\Delta = b^2 - 4ac$) and a *nested* categorical nesting. Both use exhaustive, mutually exclusive classes; both required an explicit reachability/discreteness check (ATM: which integers are multiples of 50,000; quadratic: which integers are attainable discriminants) before boundary values could be safely chosen.

Boundary Comparison. Table 26 lists the boundary types present in each case study.

Table 26: Boundary types present in each case study

Boundary type	ATM	Quadratic
Input-domain boundary	$amount \in \{50{,}000, 20{,}000{,}000\}$	$a, b, c \in \{-100, 100\}$
Invalid/valid boundary	amount divisibility	(none – all triples valid)
Logical boundary	Active/Frozen	$a = 0$ case split
Output boundary	5-way D_{out} transitions	6-way D_{out} transitions
Derived boundary	$balance = amount + fee$; $dailyUsed + amount = 20\text{M}$	$\Delta = b^2 - 4ac = 0$
Multi-variable interaction	fee×balance; two derived boundaries simultaneously	Δ crossing via any of a, b, c

Boundary Interaction Comparison. In the ATM, boundary interaction is *additive/linear* (a threshold that shifts by a constant amount, or two independent sums that can each be tight at once). In the quadratic solver, boundary interaction is *multiplicative/non-linear* (the discriminant surface is curved, so the "distance" from the boundary depends jointly and non-linearly on all three variables). Both, however, share the property that no single-variable-at-a-time test plan can reliably locate the interaction boundary; both required explicitly solving for one variable in terms of the others to construct a combination test.

Mutation Analysis Comparison. ATM mutants are dominated by relational-operator and boundary-constant mutations (reflecting its rule-based, inequality-heavy specification); quadratic mutants are dominated by arithmetic-formula mutations and classification swaps (reflecting its single-formula, case-classification specification). In both case studies, the mutants hardest to kill without a dedicated boundary test are the ones that only misbehave exactly at the =-boundary ($\Delta = 0$ for N2/N3; $balance = T$ for M1) – confirming, independently in both domains, that boundary-adjacent tests carry disproportionate fault-detection value relative to interior tests.

Executed Comparison. Executed against an implementation, the two designs behaved alike (§4.13, §5.13): both analytical matrices were reproduced cell for cell, and in both case studies the only tool-generated mutants that the designed suite did not kill were those removing a guard on inputs outside the modelled domain.

5.14.2 Lessons Common to Both Case Studies

From Partitions to Boundaries. Equivalence partitioning tells the tester *where* to sample; it says nothing about *which point within a class* to sample. Both case studies show that an arbitrarily chosen representative of a valid class (say, any *amount* multiple of 50,000, or any (a, b, c) with $\Delta > 0$) will pass, but will do so *for the wrong reason* if it happens to sit far from every boundary – it exercises the class's "obviously correct" interior, not the fragile edge where an implementation is actually likely to be wrong. BVA is therefore not an alternative to EP but its necessary refinement: EP supplies the classes, BVA supplies the points within (and between) them that carry the highest diagnostic value.

Why Single-Variable Testing Is Insufficient. Both case studies contain at least one constraint that is a genuine function of two or more variables ($balance \geq amount + fee$; $\Delta = b^2 - 4ac$). For any such constraint $\phi(x, y) = 0$, the set of points satisfying it is a lower-dimensional subset of the joint input space. Varying x alone while y is fixed at an arbitrary nominal value samples a single line through that space, and on a finite, discrete input domain that line meets the boundary curve $\phi = 0$ at only finitely many, and generically zero, values of x – a single-variable

BVA plan (which only ever tests x at *its own* domain/range boundaries) has no principled reason to land on one of them. The only way to reliably test $\phi = 0$ is to *solve* the constraint for one variable given the other(s) and construct the test tuple directly – which is by definition a boundary-combination test, not a single-variable one.

Multi-Variable Boundary Interactions. The ATM's fee interaction (§4.9, C5) and the quadratic's crossing tuple (§5.9, Q7) are structurally the same manoeuvre: hold all but one variable fixed, and show that varying the remaining one alone flips the output across a derived boundary. The pedagogical value of presenting both is that the manoeuvre generalises across a purely rule-based domain (ATM) and a purely algebraic domain (quadratic), suggesting it is a property of *derived boundaries in general*, not an artifact of either problem.

Mutation Testing as Test-Suite Evaluation. Both mutation matrices (§4.12, §5.12) show the same pattern: mutants that alter behaviour only exactly at an equality boundary ($\Delta = 0$, or $balance = T$) are killed exclusively by the corresponding boundary test and survive every interior test. Derived analytically in Sections 4 and 5 and reproduced by execution in §4.13 and §5.13, this is direct evidence for BVA's central premise – that boundary tests are not merely *additional* coverage but carry fault-detection power that interior tests structurally cannot provide for this class of fault.

Interactions, Not Isolated Inputs, Define Behaviour. Both case studies exhibit the same underlying structure once stripped of domain vocabulary: outputs are not properties of any single input in isolation but of *relations among* inputs. $balance \geq amount + fee$ is meaningless read as a statement about *balance* alone, or *amount* alone – it is a statement about their relationship, and that relationship is itself modulated by a third, categorical input (*isInternal*) that determines which relationship is even being asked. Likewise $\Delta = b^2 - 4ac$ has no meaning attached to a, b, or c individually; it is a joint property that only exists as a relation among the three. This is why a test plan cannot treat each input as an independent variable to be covered in isolation: as the comparison of boundary interactions in §5.14.1 and the single-variable argument above show, a single-variable BVA plan almost never lands on a derived boundary, and that is the direct cost of doing so. A test plan that studies each input as though it were independent of the others systematically misses these interactions; only a plan that treats the joint relation, not the individual variable, as the unit of analysis – which is exactly what the boundary-combination tests (C1–C9, Q1–Q9) are built to do – can reliably exercise them.

A One-Unit Quantitative Step Can Flip the Output Category. The second recurring pattern across both case studies is that a derived quantity can range continuously (here, in integer steps) across a wide region while the system's output classification stays constant, and then flip to a different classification at one specific step, with no intermediate category in between. Concretely: $dailyUsed + amount$ ranging over $[0, 20{,}000{,}000]$ always yields Approved-eligible output; the single step $20{,}000{,}000 \rightarrow 20{,}000{,}001$ flips it to Rejected_DailyLimit. The quadratic case shows the same pattern as a genuine multi-step transition rather than a single flip: the reachable sequence $\Delta = -3 \rightarrow 0 \rightarrow 5$ (Table 17, Q7) crosses the $\Delta = 0$ boundary, and the corresponding output sequence NoRealRoots → OneRepeatedRealRoot → TwoDistinctRealRoots changes classification twice, not once – each unit step of b (holding a, c fixed) is an equally small, uniform change to the input, yet only two of the steps in this sequence change the qualitative character of the solution set ("no real intersection with the x-axis" → "tangent to it" → "crosses it twice").

This is precisely why boundary value analysis targets these specific points rather than sampling the interior of a class: a test suite that only samples values far from every boundary can never observe one of these flips, because by construction no flip occurs there; only a test placed at or straddling the exact transition point can. This is offered as the summary intuition behind

BVA's emphasis on boundaries, not as an independent proof of it: the mutation matrices in §4.12 and §5.12 (analytically derived, then reproduced by execution in §4.13 and §5.13) are consistent with it (mutants that only misbehave exactly at an equality boundary are killed exclusively by boundary tests), but, as Section 6 notes, the mutant set analysed here was itself chosen to probe those same boundaries, so this consistency should be read as an illustration of the pattern, not as independent, unbiased evidence for it.

6 Limitations

- The mutant-killing claims in Sections 4.12 and 5.12 were derived analytically; §4.13 and §5.13 confirm them by execution against one implementation written for this report. A different implementation could deviate from the specification in ways that this implementation does not exhibit.

- The mutant sets (M1–M14, N1–N10) were themselves designed to probe the boundaries already identified in Sections 4.8–4.9 and 5.8–5.9, and the five tests added late in the ATM suite's construction (TC-ATM-18/-19/-20/-21/-22, §4.10) plus one in the quadratic suite (TC-QE-16, §5.10) were added specifically because an earlier candidate set failed to kill a particular mutant. The resulting full (13/13, 10/10) analytically expected coverage is therefore, in part, a designed-in property of this boundary-first construction process, not an unbiased test of whether boundary tests are generally superior to interior ones. The tool-generated mutants of §4.13 and §5.13 are a first population that the suites were not calibrated against, but they are few (23 and 24) and come from one short implementation; a fair comparison with interior or random suites would need larger, independently generated mutant populations over several implementations.

- *balance* was treated as an unbounded integer (currency precision and multi-day rolling-window policy beyond R3/R5 are not modelled, since none is specified); *dailyUsed* was refined to $\mathbb{Z}_{\geq 0}$ (§4.2) rather than left unbounded, on the grounds that it is a same-day accumulator of non-negative withdrawals – this is a modelling decision, not a behaviour the specification itself states, and a different accumulator implementation (e.g. one that also subtracts reversed transactions) could in principle produce a negative value that this report does not analyse.

- The quadratic solver's bounded domain $[-100, 100]^3$ is a testing convenience, not a specification requirement; the classification logic itself is domain-size-independent, so this bound affects only which domain-edge boundary tests (§5.8) are meaningful, not the discriminant analysis.

- Equivalence of M4 was argued in §4.12 and checked by execution over 20,300,001 amounts (§4.13); for every other mutant, kill status was established against the suite inputs only, first by substitution and then by execution. That a mutant is non-equivalent follows from the suite input that kills it, but no search was made for further inputs that distinguish it.

- The implementation and the executable mutant models were written by the same party that designed the suites, so the executed results test the consistency of design, models and implementation with the stated specification, not the specification's adequacy for a real ATM or solver.

- The arbitrary-precision oracle of the exhaustive sweep (§5.13) re-states the same formula as the implementation, so it detects overflow and branch-order errors but is not an independent check of the formula.

- The Microsoft Testing Platform runner of Stryker.NET was marked as a preview at the time of the run; the per-test kill attributions in §4.13 and §5.13 are taken from its reports. Timings come from a single machine and are indicative only.

7 Conclusion

This report applied the same disciplined pipeline – specification → domain → partition → boundary → boundary interaction → test → mutation – to two structurally different multi-variable systems: a rule-based financial validator and an algebraic classifier. In both, individually well-formed, per-variable boundary coverage was shown to be insufficient whenever the specification's real decision surface is a *relation* among variables rather than a predicate on any one of them; multi-variable boundary-combination tests were constructed explicitly to close that gap, and the mutation matrices confirm that the resulting suites detect exactly the faults that a naive, one-variable-at-a-time suite would miss. Executing the design confirmed it: all 166 tests passed against an implementation of both specifications, the 24 mutant columns of the two matrices were reproduced cell for cell, every tool-generated mutant inside the modelled domains was killed by the paper suites, and the exhaustive sweep showed that the $\Delta = 0$ boundary covers only 0.0137 % of the quadratic domain, so random sampling would rarely reach it. Every test case in this report answers the question "why does this test case exist?" by pointing to a specific partition, boundary, derived constraint, or interaction – which is the property that separates a designed test suite from an arbitrary list of inputs.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

The implementation, all test code, the executable mutant models, the Stryker.NET configuration and the continuous-integration workflow that re-runs the tests and the mutation analysis are kept in the source repository accompanying this report and are available from the author on request. §3.7 lists the tool versions used.

CRediT authorship contribution statement

Conceptualization, Methodology, Software, Formal analysis, Investigation, Validation, Visualization, Writing – original draft, Writing – review & editing: Mai Hải Thành.

Declaration of generative AI use

During the preparation of this manuscript, generative AI assistance (Claude, Anthropic) was used to help restructure the author's informal notes into an academic paper format, assist with the organization and drafting of the equivalence partitioning, boundary value analysis, and mutation testing sections, cross-check mathematical and test-design reasoning, and assist with language editing. For the empirical validation (§3.7, §4.13, §5.13), AI coding assistants (Claude, Anthropic, and models accessed through the OpenCode agent) were used to help write the implementation, the test code, the executable mutant models and the continuous-integration configuration, and to run the test and mutation tools; all reported numbers are taken from the

generated tool reports. The author reviewed and revised the AI-assisted material and takes full responsibility for the final content of this manuscript.

References

[1] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. John Wiley & Sons, Hoboken, NJ, 2011.

[2] B. Beizer, *Software Testing Techniques*, 2nd ed. Van Nostrand Reinhold, New York, 1990.

[3] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on Test Data Selection: Help for the Practicing Programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978.

[4] Y. Jia and M. Harman, "An Analysis and Survey of the Development of Mutation Testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[5] International Software Testing Qualifications Board (ISTQB), *Certified Tester Foundation Level Syllabus*, v4.0, 2023.